



北京大学

本科生毕业论文

题目：基于 Poseidon2 的 SPHINCS+盲签名
方案设计与实现

Design and Implementation of a
Poseidon2-Based SPHINCS+ Blind
Signature Scheme

姓 名：	刘震涛
学 号：	2200013119
院 系：	信息科学技术学院
专 业：	计算机科学与技术
导师姓名：	关志

二〇二六年五月

北京大学本科毕业论文导师评阅表

学生姓名	刘震涛	本科院系	信息科学技术学院	论文成绩	良
学生学号	2200013119	本科专业	计算机科学与技术	(等级制)	
导师姓名	关志	导师单位/ 所在学院	北京大学软件工程 国家工程研究中心	导师职称	副研究员
论文题目	中文	基于 Poseidon2 的 SPHINCS+盲签名方案设计与实现			
	英文	Design and Implementation of a Poseidon2-Based SPHINCS+ Blind Signature Scheme			
导师评语 该生毕业论文选题聚焦后量子密码与零知识证明交叉领域，围绕以 Poseidon2 为基础 SPHINCS+ 盲签名方案的设计、实现与验证展开，属于理论研究与工程实现相结合的综合课题，具有一定的前沿性、学术价值和应用意义。 论文内容涉及 SPHINCS+、Poseidon2 哈希函数、Fischlin 风格盲签名流程以及 Winterfell STARK 实现等多个方面，知识覆盖面较广，研究难度较大，工作量较为充实，达到了毕业论文应有的工作量要求。 在论文完成过程中，该生能够较好地开展文献调研、方案分析、系统设计、代码实现、实验测试和结果比较，体现出较扎实的专业基础、较强的独立思考能力和实践能力。 本课题较好地实现了对学生科研素养、工程能力和论文写作能力的综合训练，符合本专业人才培养目标要求，同意参加毕业论文答辩。 导师签名： 2026 年 5 月 14 日					

版权声明

任何收存和保管本论文各种版本的单位和个人，未经本论文作者同意，不得将本论文转借他人，亦不得随意复制、抄录、拍照或以任何方式传播。否则，引起有碍作者著作权之问题，将可能承担法律责任。

摘要

随着量子计算的发展，传统公钥密码体制面临潜在安全威胁，后量子密码迁移已成为密码学研究与应用中的重要问题。在后量子数字签名方案中，哈希基签名因其安全假设较为保守而受到广泛关注，其中 SPHINCS+ 已被 NIST 标准化为 SLH-DSA，成为无状态哈希基签名的代表方案。然而，SPHINCS+ 的标准实例化主要面向传统软件验证场景，其底层哈希结构并不天然适合零知识证明系统中的算术化表达，这限制了其在隐私保护协议、匿名凭证与可证明验证场景中的进一步应用。

针对上述问题，本文提出并实现一种面向零知识证明场景的 Poseidon2 实例化 SPHINCS+ 盲签名方案。本文贡献主要包括三个方面：第一，在技术路线层面，将 SPHINCS+ 的底层哈希接口系统性替换为更适合算术化证明的 Poseidon2，并完成域分离、参数适配、THASH 语义保持和哈希调用追踪设计；第二，在协议层面，将 Fischlin 风格盲签框架与 Poseidon2-SPHINCS+ 签名核心结合，给出承诺、签发、展示和验证四个阶段的可证明化对象关系；第三，在工程实践层面，实现 C 端签名主链与 Rust/Winterfell STARK 后端的跨语言对接，形成可运行、可测量、可复现实验的后量子隐私认证方案。

实验结果表明，本文方案能够完成 Poseidon2 实例化签名、Fischlin 风格盲签交互以及基于证明对象的展示验证流程。在传统 CPU 环境下，Poseidon2 替换版相较 SHA2 基线承担了更高的签名与验签开销；但在 STARK 电路中，Poseidon2 在 F/H 两类核心 THASH 场景下显著降低证明大小、证明时间和验证时间。因此，本文工作的主要意义在于验证了一条从 Poseidon2 哈希替换、到 Fischlin 风格盲签主链构造、再到展示证明对象落地的工程化路径，为哈希基后量子签名向零知识友好验证场景迁移提供了可复现的实现参考。

关键词：后量子密码；SPHINCS+；Poseidon2；盲签名；零知识证明；STARK

ABSTRACT

With the development of quantum computing, classical public-key cryptosystems are facing potential security threats, making the migration to post-quantum cryptography an important research and engineering problem. Among post-quantum digital signature schemes, hash-based signatures have attracted significant attention due to their conservative security assumptions, and SPHINCS+ has been standardized by NIST as SLH-DSA, becoming a representative stateless hash-based signature scheme. However, the standard instantiations of SPHINCS+ are mainly designed for conventional software verification, and their underlying hash structures are not naturally suitable for arithmetization in zero-knowledge proof systems, which limits their applicability in privacy-preserving protocols, anonymous credentials, and proof-friendly verification scenarios.

To address this problem, this thesis proposes and implements a Poseidon2-instantiated SPHINCS+ blind signature scheme for zero-knowledge proof settings. The contributions are threefold. Technically, the hash interfaces of SPHINCS+ are systematically replaced with Poseidon2, together with domain separation, parameter adaptation, THASH semantic preservation, and hash-call tracing. At the protocol level, a Fischlin-style blind-signature workflow is combined with the Poseidon2-SPHINCS+ signing core, yielding proof-oriented object relations for commitment, issuing, showing, and verification. From an engineering perspective, the C-based signature implementation is connected to a Rust/Winterfell STARK backend, producing a runnable, measurable, and reproducible post-quantum privacy-authentication scheme.

Experimental results show that the proposed scheme can complete Poseidon2-based signing, Fischlin-style blind signing, and proof-object-based show-and-verify procedures. In a conventional CPU environment, the Poseidon2 variant incurs higher signing and verification cost than the SHA2 baseline; in STARK circuits, however, Poseidon2 substantially reduces proof size, proving time, and verification time for the dominant F/H THASH relations. Therefore, the main value of this work is to validate a reproducible engineering path from Poseidon2-based hash replacement, to Fischlin-style blind-signature workflow construction, to proof-object-based showing and verification, providing an implementation reference for migrating hash-based post-quantum signatures into zero-knowledge-friendly verification scenarios.

KEY WORDS: post-quantum cryptography; SPHINCS+; Poseidon2; blind signature; zero-knowledge proof; STARK

目录

第一章 绪论	1
1.1 研究背景与意义	1
1.1.1 量子计算对现有密码体系的威胁	1
1.1.2 后量子签名方案概述：从格基到哈希基	1
1.2 SPHINCS+的优化研究现状	2
1.2.1 签名压缩优化	2
1.2.2 内存优化	2
1.2.3 参数重搜与动态优化	3
1.2.4 哈希函数替换	3
1.2.5 特定硬件实现	3
1.3 后量子零知识证明领域研究现状	3
1.3.1 零知识友好哈希函数	4
1.3.2 面向零知识证明的后量子签名研究	4
1.3.3 后量子盲签名与隐私保护研究现状	4
1.4 研究内容与贡献	5
1.4.1 研究动机	5
1.4.2 研究内容与主要贡献	5
1.5 论文组织结构	6
第二章 预备知识	7
2.1 哈希函数的密码学定义	7
2.1.1 哈希函数的基本概念	7
2.1.2 量子环境下的哈希安全属性	7
2.2 可调哈希函数	8
2.2.1 定义	8
2.2.2 安全属性	8
2.3 随机预言机模型与量子随机预言机模型	8
2.3.1 随机预言机模型（ROM）	8
2.3.2 量子随机预言机模型（QROM）	9

2.4 SPHINCS+ 签名框架	9
2.4.1 WOTS+: 一次签名方案	9
2.4.2 FORS: 少次签名方案	10
2.4.3 超树 (HyperTree) 结构	10
2.4.4 消息 M 的双遍处理问题	10
2.5 Poseidon2 零知识友好哈希函数	12
2.5.1 设计背景	12
2.5.2 Poseidon2 的构造	13
2.5.3 安全性	13
2.6 零知识证明系统	14
2.6.1 基本定义	14
2.6.2 STARK 系统	14
2.7 盲签名方案与 Fischlin 框架	15
2.7.1 盲签名的定义与安全模型	15
2.7.2 Fischlin 盲签名构造框架	15
第三章 SPHINCS+ 的 Poseidon2 哈希替换	18
3.1 设计目标与替换边界	18
3.1.1 设计动机与技术选型	18
3.1.2 替换后保留的 SPHINCS+ 语义	21
3.2 Poseidon2 后端与域分离设计	21
3.2.1 增量海绵接口	22
3.2.2 域分离标签	22
3.3 四类底层哈希接口的适配	22
3.3.1 prf_addr 接口	23
3.3.2 PRF_msg 接口	23
3.3.3 H_msg 接口	23
3.3.4 thash 接口	24
3.4 工程实现细节	24
3.4.1 字节到 Goldilocks 域元素的映射	24
3.4.2 轮函数与固定参数配置	25

3.4.3 trace 与约束统计适配.....	25
3.5 安全性讨论与表述边界.....	25
3.6 本章小结.....	26
第四章 基于 Fischlin 框架的盲签名方案设计与实现.....	27
4.1 技术选型、设计目标与符号约定.....	27
4.1.1 选择 Fischlin 框架的动机.....	27
4.1.2 选择 STARK 与 Winterfell 的动机.....	27
4.1.3 设计目标与符号约定.....	28
4.2 协议描述.....	28
4.2.1 协议总体结构.....	28
4.2.2 承诺与签发阶段.....	29
4.2.3 证明生成阶段.....	29
4.2.4 验证阶段.....	30
4.3 公开语句、私有见证与证明关系.....	30
4.3.1 公开语句.....	30
4.3.2 私有见证.....	30
4.3.3 证明关系的形式化写法.....	31
4.4 工程对象与接口实现.....	31
4.4.1 协议主链的接口分层.....	31
4.4.2 展示对象与内部凭证的结构体编码.....	32
4.4.3 最终公开对象的构造.....	32
4.4.4 公开输入与私有见证的打包.....	32
4.5 STARK 证明系统的接口与 AIR 落地.....	32
4.5.1 跨语言 FFI 边界.....	32
4.5.2 严格输入与见证关系预检.....	33
4.5.3 Trace 与公开输入的列布局.....	33
4.5.4 承诺与最终对象的证明化路径.....	33
4.5.5 证明参数配置与验证策略.....	33
4.5.6 完成边界与混合证明模型.....	33
4.6 本章小结.....	34

第五章 实验设计与结果分析	35
5.1 实验目标与实验环境	35
5.1.1 实验目标	35
5.1.2 实验环境	35
5.2 性能评价指标与实验口径	36
5.2.1 评价指标	36
5.2.2 数据口径	36
5.3 普通 CPU 环境下的传统签名与验签 benchmark	36
5.3.1 实验目的与方法	36
5.3.2 实验结果	37
5.4 Fischlin 主链的正确性与性能测试	37
5.4.1 前置正确性回归	37
5.4.2 主链与 STARK 后端的性能数据	38
5.5 面向 Poseidon2 与 STARK 的参数搜索	39
5.5.1 不能直接沿用标准参数的原因	39
5.5.2 搜索流程	39
5.5.3 实验结果	40
5.6 Poseidon2 与 SHA2 在 STARK 电路中的对比	41
5.6.1 实验目的与方法	41
5.6.2 THASH STARK 对比: F 场景	41
5.6.3 THASH STARK 对比: H 场景	41
5.6.4 F/H/T_I 调用占比	42
5.7 相关工作性能对比	43
5.8 本章小结	44
第六章 总结与展望	46
6.1 全文工作总结	46
6.2 研究展望	46
参考文献	48
附录 A 代码实现清单	53
附录 A.1: 第三章 Poseidon2 哈希替换相关代码	53

附录 A.2 第四章 Fischlin 展示证明与 STARK 接口相关代码	56
致谢	60
北京大学学位论文原创性声明和使用授权说明	61

第一章 绪论

1.1 研究背景与意义

1.1.1 量子计算对现有密码体系的威胁

数字签名作为现代信息安全基础设施的核心组件，广泛应用于电子政务、金融交易、软件分发及网络认证等各种关键领域。当前部署的主流数字签名方案包括基于大整数分解困难性的 RSA 签名、基于离散对数困难性的 DSA 签名以及基于椭圆曲线离散对数困难性的 ECDSA 签名等，其安全性均建立在经典计算模型下的数论难题之上。然而，量子计算技术的迅猛发展正在从根本上动摇这一安全基础。

1994 年，Shor 提出了量子多项式时间算法^[1]，能够在多项式时间内求解大整数分解问题与离散对数问题，这意味着一旦具备足够规模的容错量子计算机可供实际使用，上述所有基于这两类数论难题的公钥密码方案将面临崩溃的风险。此外，Grover 量子搜索算法对对称密钥方案与哈希函数亦构成平方根加速威胁^[2]，使得哈希函数的输出长度在后量子安全语境下需相应加倍。面对“先存储、后解密（store now, decrypt later）”攻击策略的现实威胁，2024 年 8 月，NIST 正式发布了后量子密码标准 FIPS 203（ML-KEM）^[3]、FIPS 204（ML-DSA）^[4]与 FIPS 205（SLH-DSA）^[5]，标志着全球密码学界已将抗量子迁移列为迫切的工程任务。

1.1.2 后量子签名方案概述：从格基到哈希基

在 NIST 后量子密码标准化项目所遴选的数字签名方案中，以格基方案为主流。CRYSTALS-Dilithium（标准化为 ML-DSA，FIPS 204）与 FALCON（标准化为 FN-DSA）分别依赖模格（Module Lattice）上的带误差学习（MLWE）问题和 NTRU 格上的短整数解（SIS）问题^[4]。格基签名方案的主要优势在于签名与公钥尺寸相对紧凑、计算效率较高；然而，其安全性依赖于特定代数结构假设（如理想格、NTRU 格等），历史上已出现参数集在新型攻击下安全界被迫向下修正的情况^[6]。

相比之下，纯哈希基签名方案的安全性仅归约至底层哈希函数的单向性、抗碰撞性与抗第二原像性，无需引入任何额外的代数结构假设。这一安全特性是哈希基签名这一技术路线的共同优势——以 XMSS 为例，IETF RFC 8391 所指出的：“XMSS provides cryptographic digital signatures without relying on the conjectured hardness of mathematical problems. Instead, it is proven that it only relies on the properties of cryptographic hash functions.”^[7]这一特性使得哈希基签名成为目前安全假设最为保守、可信度最高的后量子签名技术路线。

SPHINCS+正是这一保守路线的代表性方案。它于 2019 年由 Bernstein 等人发表于 ACM CCS^[8]，在 NIST 后量子密码标准化项目中被标准化为无状态哈希基数字签名算法（SLH-DSA，FIPS 205）^[5]。SPHINCS+采用超树（HyperTree）结合 FORS 少次签名（Few-Time

Signature) 和 WOTS+一次签名 (One-Time Signature) 的层次化架构, 实现了无状态设计, 避免了 XMSS、LMS 等有状态方案在密钥管理上的脆弱性风险。SPHINCS+的安全性最终归约至底层哈希函数的标准安全属性, 其紧致安全证明由 Barbosa、Dupressoir、Hülsing、Meijers 与 Strub 于 ASIACRYPT 2024 中采用 EasyCrypt 证明助手完成了形式化验证^[9]。

1.2 SPHINCS+的优化研究现状

SPHINCS+被 NIST 选为后量子签名标准后, 其性能优化问题受到学术界的广泛关注。现有研究从签名压缩、内存优化、哈希函数替换、参数重搜、特定硬件实现等多个维度展开, 形成了较为系统的优化研究体系。

1.2.1 签名压缩优化

SPHINCS+原始方案的主要缺陷在于签名尺寸较大 (安全级别 I 下小参数集签名约为 7856 字节), 这在带宽受限场景下构成实际瓶颈。针对这一问题, 学界提出了多种压缩方案。

SPHINCS+C (Compressing SPHINCS+ with Almost No Cost) 由 Hülsing、Kudinov、Ronen 与 Yagev 提出, 并于 2023 年 IEEE 安全与隐私研讨会 (S&P 2023) 发表^[10], 通过引入 WOTS+C 与 FORS+C 两种压缩版本的一次签名与少次签名原语, 利用特定消息子集上可进行无损压缩的代数特性, 在保持与原始 SPHINCS+等价紧致安全证明的前提下, 将 128 位安全级别下的签名尺寸从 7856 字节压缩至约 6304 字节, 实现了约 20% 的压缩比, 且签名生成时间同步降低。SPHINCS+C 的关键贡献还在于提供了签名尺寸与计算开销之间更灵活的权衡曲线, 允许针对不同应用场景独立优化目标函数。

SPHINCS- α (SPHINCS+a) 由 Zhang、Cui 与 Yu 向 NIST 补充签名标准征集项目提交^[11], 针对 WOTS+一次签名方案的编码策略进行了重新设计, 证明了所提出的编码方案在所有树状一次签名结构中具有最优尺寸, 亦优于任何有向无环图 (DAG) 结构。通过将更紧凑的一次签名方案集成于超树框架, 可在保持签名生成速度不低于 SPHINCS+的约束下实现 small 参数集的进一步签名压缩。

1.2.2 内存优化

SPHINCS+的完整签名尺寸 (最大可达约 49 KB) 与签名生成过程中的栈内存需求, 给受限嵌入式设备 (如可信平台模块 TPM、IoT 终端等) 的集成部署带来了显著困难。

Niederhagen、Roth 与 Wälde 针对 TPM 应用场景提出了流式 (Streaming) SPHINCS+ 接口, 发表于 AFRICACRYPT 2022^[12], 允许签名者在计算的同时将签名字节流式输出, 验证者以流式输入方式消费签名, 从而消除了在内存中缓存完整签名的需求。该方法在 ARM Cortex-M4 平台上的测试表明, 内存需求可显著降低, 且不引入实质性的性能开销, 使得原本因内存不足而只能使用较小参数集的设备能够完整运行所有 SPHINCS+参数变体。此类流式接口研究为 SPHINCS+在资源严格受限设备上的实用化部署提供了重要的工程基础。

1.2.3 参数重搜与动态优化

SPHINCS+的安全性及性能高度依赖于超树高度 h 、层数 d 、FORS 参数 k 、 t 以及 WOTS+ 参数 w 等一系列参数的选取。现有研究对参数空间进行了深入探索。

Kölbl 与 Philipoom 注意到, 原始参数集在最大签名次数 $q = 2^{64}$ 的保守设定下存在明显冗余^[13], 通过将 q 收缩至实际应用场景所需的规模(如 $q = 2^{20}$), 并在更大的参数空间内重新搜索最优参数集, 可在签名尺寸上取得超过 33% 的压缩效果, 同时显著提升验证速度。此外, Zhou 等提出动态优化方法^[14], 通过建模用户的签名行为模式, 在加密强度、签名速度与存储开销三者之间自适应取得最优平衡。

1.2.4 哈希函数替换

SPHINCS+的模块化设计允许将底层哈希函数替换为其他安全哈希原语, 从而为面向特定应用场景的定制化优化提供灵活性。已有研究探索了多种替换方案: Kiktenko 等人将俄罗斯标准化哈希函数 Streebog 集成于 SPHINCS+ 框架^[15], 分析了其安全性与性能影响; 在国密算法方向, 有研究将中国商用密码标准 SM3 替换进 SPHINCS+ 框架, 提出了 SPHINCS+-SM3 方案。该工作在保持 NIST PQC Level-1 安全性的前提下, 系统验证了 SPHINCS+ 哈希接口对非标准哈希函数的可扩展性, 并给出了若干参数集的性能评估结果^[16]。Cherkaoui 等人将基于 $SL_n(\mathbb{F}_p)$ 的代数哈希函数嵌入 SPHINCS+, 构造了 Spinel 签名方案^[17], 提供了与标准哈希不同的代数安全论证。这类哈希替换研究表明, SPHINCS+的超树结构对底层哈希函数具有良好的兼容性, 为面向特定性能目标(如零知识友好性)的哈希替换奠定了方法论基础。

1.2.5 特定硬件实现

针对 SPHINCS+签名生成中大量哈希计算的计算密集型特点, 多项研究致力于通过硬件加速降低时延。Amiet 等人于 2020 年首次报告了基于 FPGA 的 SPHINCS+硬件实现^[18], 在性能优化架构下相比参考软件实现实现了大幅加速, 同时发现其存在故障注入(Fault Injection)脆弱性并讨论了相应防护对策。López-Valdivieso 等人提出了基于 RISC-V 软核与 Keccak/Haraka 加速模块协同的硬件-软件协同架构^[19], 在 FPGA 平台上实现了显著的性能提升。最新工作 SPHINCSLET 由 Deshpande 等人提出^[20], 实现了首个完全符合 SLH-DSA 标准且面积高效的 SPHINCS+硬件加速器, 在 AMD Artix-7 FPGA 上针对任意安全级别仅需不超过 10.8K LUT, 较高速设计减少面积 4.7 倍, 同时比最高效的协处理器架构提速 2.5 至 5 倍。此外, 还有工作针对 GPU 并行架构^[21]和嵌入式系统^[22]探索了 SPHINCS+签名生成的编译器级优化策略。

1.3 后量子零知识证明领域研究现状

在后量子密码学与零知识证明(ZKP)的交叉领域, 如何在不依赖椭圆曲线等经典代

数结构的前提下构造高效、简洁的后量子零知识证明系统，是近年来的核心研究挑战。

1.3.1 零知识友好哈希函数

传统密码学哈希函数（如 SHA-256、SHAKE-256）设计之初以抵抗经典算法攻击为目标，其内部结构（大量布尔操作、模加等）在被表示为算术电路时产生极高的约束数量，导致在 SNARK/STARK 等零知识证明系统中证明哈希计算的开销巨大，严重制约了基于哈希的后量子方案在 ZKP 系统中的可用性^[23]。

为解决这一问题，学界提出了专为零知识证明场景设计的算术化友好哈希函数。Poseidon 由 Grassi 等人于 2021 年 USENIX Security 发表^[24]，基于 HADES 置换策略，在 GF(p) 有限域上原生操作，相比传统哈希函数在 R1CS 约束数上具有显著优势。Poseidon2 由 Grassi 等人于 2023 年提出^[25]，在 Poseidon 基础上对线性层进行了重新设计，使 Plonk 电路约束数量降低最多 70%，乘法次数降低最多 90%，成为当前不依赖查找表的最快算术化友好哈希函数。基准测试表明，以 Poseidon2 替换 SHA-256 可将 EVM 链上的相关链上成本大幅降低^[23]，Poseidon2 已被 SP1、RISC Zero 等主流 zkVM/证明栈广泛采纳。

1.3.2 面向零知识证明的后量子签名研究

后量子签名与零知识证明的结合，近年来逐渐成为隐私保护认证与链上高效验证中的重要研究方向。其主要挑战在于，多数后量子签名方案的验证过程并非为算术电路友好性而设计，因而在嵌入 SNARK 或 STARK 证明系统时通常面临较高的约束复杂度。围绕这一问题，相关研究一方面探索专门面向零知识证明优化的新型后量子签名方案，另一方面也尝试对 SPHINCS+ 等现有方案进行结构改造，以提升其在零知识场景下的适用性。

Loquat 由 Zhang、Steinfeld、Esgin、J. Liu、D. Liu 与 Ruj 提出^[26]，是这一领域的重要突破。Loquat 基于 Legendre 伪随机函数（PRF）构造了一种 SNARK 友好的后量子签名方案，其签名验证的 R1CS 约束数约为 148,000 个，比 MPC-in-the-Head 范式签名方案少 7 至 175 倍，比原始 SPHINCS+ 的约束数少 3 至 9 倍。Loquat 通过将随机预言机替换为代数哈希函数，使得签名验证过程可以高效嵌入 SNARK 证明，并在环签名与聚合签名应用中展现了显著的尺寸优势。

Sphinx-in-the-Head 由 Chen、Dong、Newton 与 Wang 提出，发表于 ACM TOPS 2024^[27]，在 MPC-in-the-Head 框架下基于 SPHINCS+ 构造了群签名方案。该工作引入了 SPHINCS+ 的变体 F-SPHINCS+ 以降低验证算法电路复杂度，同时将 XMSS 签名替换为基于新型少次签名方案 M-FORS 的群凭证，最终借助 MPCitH NIZK 实现匿名性。相关研究还探索了将 SPHINCS+ 验证嵌入通用 SNARK/STARK 证明系统的技术路径，以实现后量子签名的链上高效验证与隐私保护应用。

1.3.3 后量子盲签名与隐私保护研究现状

盲签名（Blind Signature）由 Chaum 最初提出^[28]，允许签名者在不知晓消息内容的前提下生成有效签名，是匿名凭证、电子投票与隐私保护交易等应用的核心密码原语。在后

量子环境下，经典基于 RSA 与离散对数的盲签名方案已不再安全，构造高效的后量子盲签名成为重要研究方向。

现有后量子盲签名方案主要基于格密码学构建。Hauck、Kiltz、Loss 与 Nguyen 在 CRYPTO 2020 发表了格基盲签名的系统性修订工作^[29]，发现了此前已知格基盲签名方案安全证明中的细微缺陷，并基于标准 SIS 假设在随机预言机模型下给出了可证明安全的三轮格基盲签名构造。基于同源（Isogeny）的后量子盲签名亦有研究进展，Bhounik 等人在 arXiv 2509.06127 中提出了基于 CSIDH 框架的身份基盲签名方案（CSI-IBBS）^[30]，借助超奇异椭圆曲线上群作用的计算困难性实现后量子安全，并引入诚实验证者零知识证明增强隐私保护。基于哈希的盲签名构造研究相对较少，主要挑战在于 SPHINCS+ 等哈希基方案缺乏传统盲签名所依赖的同态代数结构，难以直接套用经典盲化（blinding）技术。

Fischlin 于 CRYPTO 2006 提出了一种基于 CRS 模型的轮最优可组合盲签名框架^[31]，通过将签名方案与非交互零知识知识证明（NIZK PoK）相结合，在不依赖特定代数结构的前提下实现盲签名：用户对消息作出承诺并发送至签名者，签名者对承诺值签名并返回，用户最终提供一个非交互零知识知识证明，证明其知晓合法承诺与对应签名，从而获得对原始消息的盲签名。该框架在随机预言机模型下具有可证明安全性，且不依赖特定代数结构，为基于哈希方案的盲签名构造提供了可行路径。将群盲签名的形式化框架与此类通用变换相结合的研究亦有探索，Ghadafi 于 ACISP 2013（University of Bristol）发表了相关的形式化工作^[32]。

1.4 研究内容与贡献

1.4.1 研究动机

综合以上分析，现有后量子签名研究存在以下不足：第一，格基签名方案虽性能优越，但安全假设相对激进，在安全性保守性方面逊于哈希基方案；第二，现有 SPHINCS+ 在零知识证明场景下效率低下，其底层哈希函数（SHA-256、SHAKE-256）在算术化电路中产生大量约束，严重制约其在后量子隐私保护应用中的可用性；第三，基于哈希的后量子盲签名方案研究仍较少，现有方案存在依赖格代数结构，以及在实用性上不足的问题。

面向后量子隐私保护这一新兴应用领域，本文选择以 SPHINCS+ 为基础框架，对其底层哈希函数进行零知识友好性优化，并在此基础上构建兼具后量子安全性与隐私保护能力的盲签名方案，以弥补上述研究空白。

1.4.2 研究内容与主要贡献

本文的主要贡献如下：

贡献一：Poseidon2 替换 SPHINCS+ 底层哈希函数。本文在 SPHINCS+ 官方参考实现代码库基础上，系统性地将底层哈希函数由 SHA-256/SHAKE-256 替换为 Poseidon2 算术化友好哈希函数。Poseidon2 在 $GF(p)$ 域上原生操作，相比 SHA-256 在 STARK 电路中的约

束数量减少约一个量级，为后续零知识证明集成奠定高效的算术化基础。本文给出了参数映射、工程化安全讨论、实现细节与性能评估。

贡献二：基于 Fischlin 框架的 SPHINCS+盲签名设计。 本文将 Fischlin 盲签名构造框架应用于基于 Poseidon2 的 SPHINCS+变体，设计并实现了基于 Fischlin 框架的 SPHINCS+ 盲签名方案。

贡献三：在工程实践层面，在 SPHINCS+官方仓库基础上实现了完整的盲签名协议框架代码，在混合证明模型下完成展示验证原型，涵盖密钥生成、盲化、签名与验证等全流程，并提供了多维度对比基准测试结果。

1.5 论文组织结构

本文共分六章，结构安排如下：

第一章（绪论）：介绍量子威胁背景与后量子密码研究现状，综述 SPHINCS+的优化研究与后量子零知识证明领域的进展，阐明研究动机与主要贡献。

第二章（预备知识）：系统介绍本文所需密码学基础，包括哈希函数安全属性、WOTS+/FORS/SPHINCS+框架、Poseidon2 哈希函数构造原理、零知识证明基础定义，以及 Fischlin 盲签名框架的形式化定义与安全模型。

第三章（SPHINCS+的 Poseidon2 哈希替换）：详细阐述将 Poseidon2 集成于 SPHINCS+框架的技术方案，包括替换边界、接口适配、域分离设计。

第四章（基于 Fischlin 框架的盲签名方案）：给出盲签名方案的设计，包括协议描述、关系定义、工程化正确性与安全语义讨论。

第五章（实验设计与结果分析）：进行功能验证、参数搜索、性能评估与证明成本实验。

第六章（总结与展望）：总结本文工作，讨论现有局限性，并展望未来研究方向。

第二章 预备知识

本章系统介绍后续章节所需的密码学基础知识，内容涵盖哈希函数的密码学定义与安全属性、可调哈希函数（Tweakable Hash Functions）、随机预言机模型（ROM）与量子随机预言机模型（QROM）、SPHINCS+签名框架的层次化结构、Poseidon2 零知识友好哈希函数的设计原理、零知识证明系统（以 STARK 为核心），以及盲签名方案的形式化定义与 Fischlin 通用构造框架。

2.1 哈希函数的密码学定义

2.1.1 哈希函数的基本概念

哈希函数（Hash Function）是现代密码学的基础原语之一。一个密码学哈希函数 $\mathcal{H}$ 是从任意长度（或固定长度）的二进制字符串到固定长度输出的映射，通常表示为：

$$\mathcal{H}: \{0,1\}^* \rightarrow \{0,1\}^n$$

其中 n 为输出位长度，称为摘要长度。密码学哈希函数的安全性通常由三类核心安全属性刻画^[33]：

单向性（One-Wayness / Preimage Resistance）：对于均匀随机选取的输出 $y = \mathcal{H}(x)$ 任意概率多项式时间（PPT）对手 $\mathcal{A}$ 在不知道 x 的情况下，找到任意 x' 使得 $\mathcal{H}(x') = y$ 的概率均为可忽略量（negligible）。

抗碰撞性（Collision Resistance）：任意 PPT 对手找到满足 $\mathcal{H}(x_1) = \mathcal{H}(x_2)$ 且 $x_1 \neq x_2$ 的一对 (x_1, x_2) 的概率均为可忽略量。

抗第二原像性（Second Preimage Resistance）：给定任意输入 x ，任意 PPT 对手找到 $x' \neq x$ 使得 $\mathcal{H}(x') = \mathcal{H}(x)$ 的概率均为可忽略量。

2.1.2 量子环境下的哈希安全属性

在量子计算环境下，哈希函数的安全边界需要相应调整。Grover 算法可以对大小为 N 的搜索空间实现 $O(\sqrt{N})$ 查询复杂度的量子加速^[2]，其对哈希函数的主要影响如下：针对单向性的暴力搜索攻击从经典 $O(2^n)$ 降至量子 $O(2^{n/2})$ ，对抗碰撞性的生日攻击复杂度亦相应下降。为在后量子环境下保持 λ 位安全级别，哈希函数输出长度至少需设置为 2λ 位（针对单向性）。此外，Hülsing、Rijneveld 与 Song 在研究多目标哈希安全属性时^[34]，引入了多目标第二原像抵抗（MT-SPR）与多目标单向性（MT-OW）等概念，为哈希基签名在多用户/多签名场景下的量子安全分析提供了精确的复杂度估计工具。

2.2 可调哈希函数

2.2.1 定义

可调哈希函数（Tweakable Hash Function, THF）是 SPHINCS+ 框架的核心抽象工具，由 Bernstein 等人在 SPHINCS+ 论文中正式引入^[8]。其形式化定义如下：

设 $\mathcal{P}$ 、 $\mathcal{T}$ 、 $\mathcal{M}$ 与 $\mathcal{MD}$ 分别为公共参数空间、调整值（Tweak）空间、消息空间与摘要空间。一个可调哈希函数族 Th 为一映射族：

$$\text{Th}: \mathcal{P} \times \mathcal{T} \times \mathcal{M} \rightarrow \mathcal{MD}$$

其中 $P \in \mathcal{P}$ 为每密钥对绑定一次的公共种子， $T \in \mathcal{T}$ 为调整值（每次哈希调用唯一）， $M \in \mathcal{M}$ 为输入消息。

2.2.2 安全属性

在 SPHINCS+ 的安全分析中，可调哈希函数主要通过以下两类性质进行刻画^[8]：

PQ-SM-TCR：即后量子、单函数、多目标、针对不同 tweak 的目标碰撞抗性。该性质要求攻击者即使面对同一公共参数下的多个目标实例，也难以针对某个给定的、互不重复的 tweak 构造满足条件的目标碰撞。

PQ-SM-DSPR：即后量子、单函数、多目标、针对不同 tweak 的判定式第二原像抗性。该性质关注的是在多目标场景下，攻击者区分或构造有效第二原像的能力应当是可忽略的。

这两类性质共同构成了 SPHINCS+ 中可调哈希函数安全分析的基础。通过在公共种子之外引入 tweak 参数，SPHINCS+ 能够将 WOTS+、FORS 与 hypertree 中不同层次的哈希调用统一到同一分析框架之下，从而支持模块化的安全归约证明。

2.3 随机预言机模型与量子随机预言机模型

2.3.1 随机预言机模型（ROM）

随机预言机模型（Random Oracle Model, ROM）由 Bellare 与 Rogaway 于 1993 年提出^[35]。在该模型中，哈希函数 $\mathcal{H}$ 被理想化为一个公共可访问的随机预言机：对于每个首次出现的输入，预言机独立且均匀地返回一个固定长度的随机输出；对于重复输入，则始终返回此前给定的同一结果。

ROM 为密码方案的可证明安全分析提供了强有力的工具。在安全归约中，证明者通常可以通过“编程”随机预言机的回答，将底层困难问题实例嵌入到对手可见的交互过程中，从而构造出从破坏密码方案到求解基础困难问题的归约证明^[35]。

ROM 的根本局限在于其理想化假设。现实系统中实际部署的是具体哈希函数，而非真正的随机预言机；因此，标准模型下确实存在这样的反例：某些方案在 ROM 中可证明

安全，但在将预言机实例化为具体哈希函数后却不再安全^[35]。尽管如此，ROM 由于能够在较强效率与较清晰证明结构之间取得平衡，仍然是现代密码学中最常用的安全证明模型之一^[35]。

2.3.2 量子随机预言机模型（QROM）

在后量子安全分析中，传统 ROM 还不足以完整刻画量子对手的能力，因此需要引入量子随机预言机模型（Quantum Random Oracle Model, QROM）^[36]。Boneh、Dagdelen、Fischlin、Lehmann、Schaffner 与 Zhandry 在 ASIACRYPT 2011 中系统讨论了这一模型^[36]。与 ROM 不同，QROM 允许对手以量子叠加态形式访问随机预言机，即对手可以对输入的叠加态发起查询，并获得相应的叠加态输出，而不必退化为某一次经典查询。

这一差异令在 QROM 中许多经典归约技术不再能够直接使用。特别是，传统 ROM 证明中常见的回溯、查询记录与条件重编程等方法，在面对量子叠加查询时会受限制，因此必须发展专门的量子归约技术^[36]。

SPHINCS+ 的安全分析正是在这一背景下展开的。其安全证明需要在 QROM 中对底层哈希相关原语进行统一建模，并借助抗量子的可调哈希安全性质来支撑整体归约^[34]。此外，Barbosa 等人的形式化验证工作进一步在 EasyCrypt 证明助手中对 SPHINCS+ 的相关量子安全归约进行了机器化验证^[9]。

2.4 SPHINCS+ 签名框架

SPHINCS+ 是一个无状态哈希基数字签名框架，由 Bernstein、Hülsing、Kölbl、Niederhagen、Rijneveld 与 Schwabe 于 2019 年发表^[8]，并被 NIST 标准化为 FIPS 205 (SLH-DSA)^[5]。其整体安全性归约至底层哈希函数的可调哈希安全属性，无需依赖任何代数困难性假设。

2.4.1 WOTS+：一次签名方案

WOTS+（Winternitz One-Time Signature Plus）由 Hülsing 于 AFRICACRYPT 2013 提出^[37]，是 SPHINCS+ 的基础签名原语。设安全参数为 n ，Winternitz 参数为 w ，WOTS+ 私钥由 $\ell = \ell_1 + \ell_2$ 个 n 字节随机值组成，其中：

$$\ell_1 = \left\lceil \frac{8n}{\log_2 w} \right\rceil, \quad \ell_2 = \left\lceil \frac{\log_2(\ell_1(w-1))}{\log_2 w} \right\rceil + 1$$

公钥通过对私钥链式应用哈希函数 $w-1$ 次计算得到：

$$\text{pk}_i = \mathcal{H}^{w-1}(\text{sk}_i), \quad i = 1, \dots, \ell$$

对消息摘要 M 进行编码后，签名通过对每个私钥值应用 b_i 次哈希函数生成（ b_i 由消息各分组值决定），验证者则通过补充剩余的哈希迭代将签名恢复为公钥。WOTS+ 的安全性在标准模型下被严格证明等价于底层哈希函数的抗第二原像性，并具有精确而紧致的

安全归约^[37]。由于 WOTS+ 是一次性方案，SPHINCS+ 通过上层树状结构实现对 WOTS+ 的托管与安全重用。

2.4.2 FORS：少次签名方案

FORS (Forest Of Random Subsets) 是 SPHINCS+ 中用于签名消息摘要前 ka 位的少次签名 (Few-Time Signature, FTS) 方案，是 SPHINCS+ 区别于其前身 SPHINCS 的关键新组件^[8]。FORS 由 k 棵高度为 a 的二叉 Merkle 树组成，每棵树有 $t = 2^a$ 个叶节点，叶节点值由秘密种子 $SK.seed$ 通过伪随机函数派生。

对一个 ka 位的消息进行签名时，将消息分为 k 段，每段值 v_i ($0 \leq v_i < t$) 对应第 i 棵树中揭露叶节点 $sk[i][v_i]$ 与其到根节点的认证路径 (Authentication Path)。公钥为 k 棵树根节点的哈希串联值。相比 SPHINCS 中使用的 HORST 方案，FORS 通过调整 k 与 t 的参数约束，可在指数级降低签名次数预算 q 时保持可证明的安全级别，从而使 SPHINCS+ 的签名次数上限达到 2^{64} ^[8]。

2.4.3 超树 (HyperTree) 结构

SPHINCS+ 通过 d 层 XMSS 树 (每层 h/d 高度) 的层次化组合构成超树 (HyperTree)，总树高为 h ，对应 2^h 个潜在的 FORS 密钥对索引^[8]。

具体层次结构如下：

第 d 层 (最顶层)：包含一棵高度为 h/d 的 XMSS 树，根节点哈希即为 SPHINCS+ 的公钥的一部分 $PK.root$ 。

第 1 至 $d-1$ 层：每层包含若干棵高度为 h/d 的 XMSS 树，上层 XMSS 树的每个叶节点对应一个 WOTS+ 密钥对，其中该叶对应的 WOTS+ 私钥用于签署下层 XMSS 子树的根节点，而该 WOTS+ 公钥经叶节点生成过程形成上层 XMSS 树的叶节点值。

第 0 层 (最底层)：每个叶节点对应一个 FORS 密钥对。

SPHINCS+ 完整签名由随机化值 R 、FORS 签名 σ_{FORS} 、以及 d 层 XMSS 签名 $(\sigma_{XMSS}^{(1)}, \dots, \sigma_{XMSS}^{(d)})$ 串联组成。参数集以安全级别 n 、树高 h 、层数 d 、FORS 参数 (k, a) 、Winternitz 参数 w 共同决定签名尺寸与性能特征。

2.4.4 消息 M 的双遍处理问题

2.4.4.1 算法流程与双遍处理的成因

SPHINCS+ 标准化为 SLH-DSA (FIPS 205) 后，其内部签名算法 “slh_sign_internal” (Algorithm 19) 的计算流程揭示了一个关键的工程约束——消息 M 必须被完整读取两次^[5]。具体而言，签名过程依次执行以下两步：

步骤一 (随机化值生成)：

$$R = \text{PRF_msg}(SK.prf, \text{opt_rand}, M)$$

步骤二 (消息摘要计算)：

$$\text{digest} = H_msg(R, PK.seed, PK.root, M)$$

步骤二在计算消息摘要时需要将步骤一得到的 R 作为输入，而 R 本身又依赖完整的消息 M ，导致 M 在整个签名过程中被完整遍历两次，即所谓的“双遍处理(double-pass)”问题。这一设计与广泛应用于密码学库的 init-update-final 流式分块接口（如 EdDSA 实现中同样存在的类似问题）存在根本性兼容障碍，并对大消息场景下嵌入式设备与硬件安全模块（HSM）的内存管理带来显著压力^[8]。

2.4.4.2 R 对 M 依赖的安全必要性

R 的计算中绑定消息 M 并非偶然的设计选择，而是出于三方面明确的安全考量^{[5][8]}：

（1）防止预计算攻击。 R 必须对攻击者不可预测。若 R 的计算中不包含 M ，则攻击者可在获取目标消息之前预先计算部分签名结构（如超树路径等），为后续选择性伪造提供便利。将 M 绑定至 R 的计算，使得任何预计算必须在知悉完整消息后方能进行。

（2）确定性模式下的消息区分。在确定性签名场景中， opt_rand 可固定为公钥种子 $PK.seed$ 。若 R 的计算不包含 M ，则同一密钥对所有消息将共享相同的 R ，极大削弱 FORS 签名的安全裕度——攻击者只需诱导签名者对若干散列到同一 FORS 叶节点的消息签名，便可通过较低的计算代价重建私钥分量。

（3）对冲签名设计（Hedged Signing）。结合 $SK.prf$ 的保密性、“ opt_rand ”的随机化以及 M 的完整绑定，形成了层次化的安全保障：即便随机数生成器（RNG）存在质量缺陷甚至后门，PRF 对 M 的绑定仍可阻止攻击者在不知晓完整消息的情况下操纵 R 。

上述设计在密码学上体现了随机化哈希（Randomized Hashing）的核心思想：将签名方案的安全性从依赖哈希函数的完全抗碰撞性（Collision Resistance, CR）降低至更弱的目标碰撞抵抗性（Target Collision Resistance, TCR）^[38]。Halevi 与 Krawczyk 在 CRYPTO 2006 的奠基性工作中指出，引入签名者选择的随机盐值后，攻击者的碰撞寻找任务从离线模式（不依赖签名者的预计算）转变为在线模式（必须与签名者交互），复杂度大幅提升^[38]。SLH-DSA 将 R 绑定至 M 正是这一原则在哈希基签名中的具体实例化： H_msg 中包含 R 与 $PK.root$ ，使消息摘要的计算同时绑定了签名者身份与随机化值，对外部碰撞攻击具有较强抵御能力。

2.4.4.3 Pure 模式与 Pre-Hash 模式

FIPS 205 为 SLH-DSA 定义了两种外部签名接口，两者共享同一内部算法 slh_sign_internal ，其核心区别在于传递给内部算法的消息构造方式不同^[5]。

Pure SLH-DSA（Algorithm 22）直接将原始消息 M 拼接于构造的消息串 M' 中：

$$M' = \text{toByte}(0,1) \parallel \text{toByte}(|ctx|,1) \parallel ctx \parallel M$$

其中域分离字节“0x00”标识 Pure 模式，“ ctx ”为上下文字符串。由于 M 作为完整原始消息拼入 M' ，签名过程不可避免地产生双遍处理问题。Pure 模式的安全分析仅依赖 TCR 而非 CR，是安全假设最为保守的选择^[8]。

HashSLH-DSA (Algorithm 23) 在传入内部算法前预先对消息进行哈希：

$$M' = \text{toByte}(1,1) \parallel \text{toByte}(|\text{ctx}|,1) \parallel \text{ctx} \parallel \text{OID} \parallel \text{PH}(M)$$

其中 $\text{PH}(M)$ 为消息的预哈希值（支持 SHA-256、SHA-512、SHAKE128 及 SHAKE256），“OID” 标识所使用的哈希函数。由于 $\text{PH}(M)$ 为固定长度， M' 的长度不再依赖原始消息规模，从而消除了双遍处理问题，与 “init-update-final” 流式接口兼容。然而，HashSLH-DSA 的安全性依赖于预散列函数 PH 的完全抗碰撞性（CR）——若 PH 的碰撞被攻破，则可在不同公钥之间复用碰撞对，削弱多用户安全性^[5]。

两种模式的主要对比如下表所示：

表 2.1 pure 与 pre-hash 对比

比较维度	Pure SLH-DSA	Hash SLH-DSA
消息遍历次数	2 次（double-pass）	1 次（single-pass）
流式接口兼容	不兼容	兼容
安全假设	TCR（更保守）	CR（更强依赖）
多用户安全	强（PK 绑定于摘要）	较弱（碰撞可跨密钥复用）

综上所述，消息 M 的双遍处理问题是 SPHINCS+/SLH-DSA 签名流程中一个由安全设计主动引入的工程约束： R 对 M 的绑定在防预计算攻击、消息区分与对冲签名三个维度上均具有明确的安全必要性，不可简单消除。FIPS 205 通过引入 Pure 模式与 HashSLH-DSA 两种接口，在保守安全假设与流式接口兼容性之间提供了显式的权衡选项。本文后续章节在进行 Poseidon2 哈希替换时，将沿用 Pure 模式语义，以保持最保守的安全假设，并在接口适配中正视双遍处理带来的工程约束。

2.5 Poseidon2 零知识友好哈希函数

2.5.1 设计背景

传统密码学哈希函数（如 SHA-256、SHAKE-256）的内部结构在表示为算术电路（R1CS、PLONK 约束系统等）时产生极高的约束数量。例如，SHA-256 的单次压缩在 R1CS 中约产生 27,000 至 30,000 个约束，导致零知识证明的证明生成开销随哈希调用次数线性增长^[23]。这使得将 SPHINCS+ 等哈希密集型方案纳入 ZK 证明电路在实践中极为困难，是后量子签名与零知识证明融合的核心瓶颈。

为此，学界开发了专为有限域上算术运算设计的算术化友好哈希函数（Arithmetization-Friendly Hash Function）。Poseidon 由 Grassi 等人于 2021 年 USENIX Security 发表^[24]，基于 HADES（Half-And-roundS with aDd-round-key, mix-layer, Equation-based sBoxes）置换框

架，在素数域 $\mathbb{F}_p$ 上原生操作。Poseidon2 由 Grassi、Khovratovich 与 Schofnegger 于 AFRICACRYPT 2023 进一步提出^[25]，在 Poseidon 基础上对线性层进行重新设计，显著降低了约束密度，成为目前性能最优的算术化友好哈希函数之一。

2.5.2 Poseidon2 的构造

Poseidon2 是一种面向算术化约束系统优化设计的哈希原语，其内部置换延续了 HADES 设计思想，整体结构由前半段全轮、中间部分轮和后半段全轮三部分组成。设状态宽度为 t ，全轮轮数为 R_F ，部分轮轮数为 R_P ，S 盒函数为幂映射 $x \mapsto x^d$ ，其中 d 通常取 5 或 7，且满足 $\gcd(d, p-1) = 1$ ^[25]。则其置换结构可表示为：先执行 $R_F/2$ 轮全轮，再执行 R_P 轮部分轮，最后执行 $R_F/2$ 轮全轮^[25]。

Poseidon2 的核心优化体现在置换内部线性层的重新设计。与原始 Poseidon 在各轮中使用稠密 MDS 矩阵不同，Poseidon2 分别为外部轮与内部轮引入了更适于高效实现的结构化线性层，以降低矩阵乘法带来的运算与约束开销^[25]。其中，外部轮使用结构化矩阵 M_E ，内部轮使用“对角矩阵加低秩修正”的矩阵 M_I ，从而使线性层能够在线性复杂度下实现^[25]。此外，Poseidon2 在置换起始位置增加了一次额外线性层，并对内部轮常数注入方式进行了简化，以进一步改善整体实现效率^[25]。

得益于上述设计，Poseidon2 在线性层相关运算中显著减少了乘法次数与电路约束数量。文献给出的结果表明，在 Plonk 等约束系统中，Poseidon2 的线性层约束数相较于 Poseidon 最多可降低约 70%，乘法次数最多可降低约 90%^[25]。因此，Poseidon2 特别适用于零知识证明场景下的哈希计算，并可按海绵（sponge）模式处理可变长度输入，也可按压缩函数（compression function）模式处理固定长度输入^[25]。

2.5.3 安全性

Poseidon2 的安全性分析主要围绕代数攻击与经典密码分析方法展开，包括 Gröbner 基攻击、XL 类攻击、插值攻击，以及差分分析和线性分析等。其参数设置目标是在保持较低算术复杂度的同时，使上述攻击在给定安全级别下的复杂度仍然足够高，从而满足单向性、抗碰撞性及置换安全性的要求^[25]。

在构造层面，Poseidon2 的海绵模式与压缩函数模式均建立在其底层置换安全性的基础之上。当所选参数能够抵抗已知的代数与统计分析方法时，基于该置换构造的哈希函数即可在相应模型下提供预期的安全保证。因此，Poseidon2 的安全性应结合具体状态宽度、轮数配置、S 盒指数及底层有限域参数进行整体评估，而不能脱离具体实例单独讨论^[25]。

当 Poseidon2 被用于替换 SPHINCS+ 中的原始哈希原语时，其安全性要求不仅限于一般意义下的抗碰撞性与单向性。由于 SPHINCS+ 的安全证明依赖于底层哈希在可调哈希框架下满足相应的多目标安全性质，因此 Poseidon2 的具体实例化还需进一步验证其是否能够支撑 SPHINCS+ 所要求的可调哈希安全属性。

2.6 零知识证明系统

2.6.1 基本定义

零知识证明系统(Zero-Knowledge Proof System)的概念由 Goldwasser、Micali 与 Rackoff 于 1989 年发表的奠基性工作正式确立^[39]。形式上,对于一个语言 $L \in \text{NP}$ (其关系为 $\mathcal{R}_L$), 一个交互式证明系统 (P, V) 是关于 L 的零知识证明, 当且仅当以下三个属性同时成立:

完备性 (Completeness): 对任意 $(x, w) \in \mathcal{R}_L$, 诚实证明者 $P(x, w)$ 与验证者 $V(x)$ 交互后, 验证者以压倒性概率接受。

可靠性 (Soundness): 对任意 $x \notin L$ 与任意恶意证明者 P^* , 验证者接受的概率为可忽略量。

零知识性 (Zero-Knowledgeness): 存在概率多项式时间模拟器 $\mathcal{S}$, 对任意 $(x, w) \in \mathcal{R}_L$ 与任意验证者 V^* , $\mathcal{S}(x)$ 生成的视图与真实交互视图在计算上不可区分。

若可靠性满足更强的知识可靠性 (Knowledge Soundness) ——存在知识提取器 (Knowledge Extractor) 能够从接受的证明中高效提取出有效见证 w ——则该系统称为知识的证明 (Proof of Knowledge, PoK); 若其为非交互式, 则称为非交互式零知识知识证明 (NIZK PoK), 简洁非交互零知识证明 (zkSNARK/zkSTARK) 作为上述框架的高效实例化方向, 在证明长度与验证效率方面已有系统性研究^[40]。

2.6.2 STARK 系统

STARK (Scalable Transparent Argument of Knowledge) 由 Ben-Sasson、Bentov、Horesh 与 Riabzev 于 2019 年 CRYPTO 正式发表^[41], 是目前最重要的后量子零知识证明系统之一, 其安全性基于哈希函数的抗碰撞性, 不依赖任何代数困难性假设。STARK 的核心技术组件如下:

(1) 算术中间表示 (AIR): 待证计算被表示为一张由满足递推约束的轨迹多项式组成的有限域表格, 每一列对应计算中的一个变量, 每一行对应一个时钟周期, 相邻行之间由低次多项式约束关联。

(2) FRI 协议 (Fast Reed-Solomon IOP of Proximity): STARK 的核心多项式承诺与低度检验协议, 由同一团队于 ICALP 2018 提出^[42], 通过交互折叠 (Folding) 将低度多项式的成员测试归约为更小规模的低度检验, 实现了严格线性的证明者复杂度与严格对数的验证者复杂度。STARK 通过 Fiat-Shamir 变换将基于 FRI 的交互式证明转化为非交互式论证。

(3) 后量子安全性与透明性: STARK 无需可信初始化 (Trusted Setup), 避免了"有毒废料 (Toxic Waste)"问题; 其安全性完全基于哈希函数的标准密码学假设, 在量子计算模型下依然安全^[42], 相比基于配对的 zkSNARK (如 Groth16) 在后量子语境下具有本质优势。STARK 的主要权衡在于证明尺寸相对较大 (通常为数十至数百 KB), 而证明生成时间与验证时间均呈准线性规模关系。

在本文所构造的盲签名方案中，STARK 作为 NIZK PoK 的实例化选择，用于证明展示者（用户）知晓合法的 SPHINCS+ 签名与对应承诺的随机性，是实现盲化性质的关键技术手段。

2.7 盲签名方案与 Fischlin 框架

2.7.1 盲签名的定义与安全模型

盲签名（Blind Signature）由 Chaum 于 1983 年提出^[28]，作为实现不可追踪电子支付系统的密码原语。盲签名方案允许用户（请求者）获得签名者对某消息 m 的有效签名，而签名者在整个协议执行过程中无法获知消息 m 的任何信息。

形式上，一个盲签名方案 $\Pi_{BS} = (\text{KeyGen}, \langle U, S \rangle, \text{Verify})$ 包含三类算法：

$\text{KeyGen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$ ：密钥生成算法，生成签名者的公私钥对。

$\langle U(m, \text{pk}), S(\text{sk}) \rangle$ ：用户与签名者之间的两方交互协议，协议结束后用户获得签名 σ ，签名者无输出。

$\text{Verify}(\text{pk}, m, \sigma) \rightarrow \{0, 1\}$ ：验证算法。

盲签名方案的安全模型包含以下两项核心安全属性^[30]：

一次多伪造不可伪造性（One-More Unforgeability）：任意 PPT 对手即便在与签名者完成至多 q 次盲签协议交互后，也无法产生 $q + 1$ 个有效签名-消息对，即对手无法利用已有交互获得“额外”的有效签名。

盲化性（Blindness）：签名者在交互结束后无法将所签署的签名与某次具体的签名协议执行实例相关联，即签名者视角下的交互记录与最终揭露的消息-签名对之间是不可区分的。

2.7.2 Fischlin 盲签名构造框架

Fischlin 于 2006 年 CRYPTO 提出了一种基于公共参考串（Common Reference String, CRS）模型的轮最优可组合盲签名通用构造。^[31] 该框架的基本思想是：用户首先对消息进行承诺，签名者再对该承诺值执行底层数字签名，随后用户利用非交互式零知识证明将“自己知晓某个消息承诺及其有效签名”这一事实封装为可公开验证的最终盲签名。与依赖特殊代数结构的盲签名方案不同，Fischlin 框架在抽象层面上并不要求底层签名方案具有配对、线性同态或可重随机化等额外性质，因此可作为一类较通用的上层构造思路。Fischlin 框架的协议流程如下：

设底层数字签名方案为：

$$\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify}),$$

承诺方案为：

$$\text{Com}: \mathcal{M} \times \mathcal{R} \rightarrow \mathcal{C},$$

非交互式零知识知识证明系统记为：

$$\Pi = (P, V),$$

其中 P 和 V 分别表示证明算法与验证算法。系统初始化时，依据安全参数 λ 生成公共参数

$$crs \leftarrow Setup(1^\lambda),$$

签名者运行

$$(pk, sk) \leftarrow KeyGen(1^\lambda)$$

生成公私钥对。

Fischlin 框架的协议流程可表述如下。

(1) 盲化阶段：用户对消息 m 选取随机数 r ，计算承诺

$$c = Com(m; r),$$

并将承诺值 c 发送给签名者。

(2) 签名阶段：签名者在收到承诺值 c 后，利用底层签名算法计算

$$\sigma \leftarrow Sign(sk, c),$$

并将签名 σ 返回给用户。

(3) 证明生成阶段：用户在收到 σ 后，首先验证

$$Verify(pk, c, \sigma) \stackrel{?}{=} 1.$$

若验证失败，则中止协议；否则，用户进一步构造最终签名载体 c^* ，并生成非交互式零知识证明

$$\pi \leftarrow P(crs, x, w),$$

其中公开语句 x 至少包含消息 m 、公钥 pk 以及最终公开值 c^* ，见证 w 包含中间承诺 c 、承诺随机数 r 以及底层签名 σ 。该证明所刻画的关系可以写为：证明者知晓某个见证 (c, r, σ) ，使得

$$c = Com(m; r) \quad \wedge \quad Verify(pk, c, \sigma) = 1 \quad \wedge \quad c^* \text{ 与 } (m, c, \sigma) \text{ 满足预定的公开关联关系}$$

因此，最终输出的盲签名可记为

$$\sigma_{blind} = (c^*, \pi)$$

这种写法更贴近 Fischlin 框架的本质，即最终签名并非直接公开底层签名 σ ，而是公开一个承载关系的对象及其对应的零知识证明。

(4) 验证阶段：验证者在输入消息 m 、盲签名 $\sigma_{blind} = (c^*, \pi)$ 、公钥 pk 以及系统公

共参数 crs 后，运行验证算法检查

$$V(crs, x, \pi) \stackrel{?}{=} 1$$

若验证通过，则接受该签名；否则拒绝。换言之，验证者验证的并不是某个显式公开的底层签名，而是证明 π 是否成功证明了：存在某个承诺值 c 与签名 σ ，满足 c 是对消息 m 的有效承诺， σ 是对 c 的有效底层签名，且它们与公开值 c^* 保持一致。

在安全性方面，该框架在 CRS 模型下实现了轮最优的并发安全与可组合安全盲签名构造。对于本文而言，Fischlin 框架的意义在于提供了“先对中间对象签名，再通过零知识证明封装为最终签名”的通用设计范式。因而，若将 SPHINCS+ 作为底层签名方案，并结合 Poseidon2 的算术化友好性，进一步借助 STARK 系统对相关关系进行高效证明，则可形成一种面向哈希基签名场景的具体实例化路线。

第三章 SPHINCS+的 Poseidon2 哈希替换

本章围绕 SPHINCS+的 Poseidon2 哈希替换展开，即在保持 SPHINCS+上层签名结构基本不变的前提下，将其底层哈希接口替换为面向零知识证明友好的 Poseidon2 后端。与重新设计一种签名算法不同，本文的目标是在尽可能保留 SPHINCS+寻址机制、密钥材料布局、签名流程语义和验证流程的基础上，完成底层哈希实例化的工程替换，并在第五节给出适合 Poseidon2 与 STARK 证明场景的参数选择方法。

本章首先说明替换设计的总体原则与边界，然后分别给出 `prf_addr`、`PRF_msg`、`H_msg` 和 `thash` 四类底层接口的 Poseidon2 适配方式；随后介绍 Poseidon2 后端的海绵接口、域分离和字节到有限域元素的映射。

3.1 设计目标与替换边界

SPHINCS+的安全性和模块化结构高度依赖底层哈希原语。其签名流程中，地址相关伪随机函数用于派生 WOTS+和 FORS 节点秘密值，消息随机化函数用于生成签名随机值 R ，消息哈希函数用于将消息映射到 FORS 摘要、子树索引和叶子索引，而树哈希函数则负责实现 F 、 H 、 T_i 等可调哈希压缩操作。因此，将底层哈希替换为 Poseidon2 时，不能简单地把所有哈希调用改写为同一个普通哈希函数，而必须保持各接口之间的语义边界。

本文实现遵循“上层结构不变、底层后端替换”的原则。具体而言，超树层数、FORS 结构、WOTS+链式计算方式、ADRS 地址编码、`public seed` 参与方式、私钥种子使用方式以及签名输出结构均保持 SPHINCS+原有语义；变化主要集中在哈希后端，即将 SHA2/SHAKE 风格的字节哈希接口替换为基于 Goldilocks 域的 Poseidon2 海绵接口，并用显式域分离标签区分不同调用语义，如图 3.1 所示。

3.1.1 设计动机与技术选型

将 SPHINCS+ 的底层哈希替换为 Poseidon2 的主要动机，来自本文研究对象与标准 SPHINCS+ 原始应用场景之间的差异。标准化的 SLH-DSA 以 SPHINCS+ 为基础，其安全性主要依赖哈希函数的抗原像、抗碰撞以及相关性质^[5]；原始 SPHINCS+ 框架也通过 WOTS+、FORS 和超树结构，将签名安全性建立在对称密码原语之上^[8]。然而，在本文的盲签名与 STARK 证明场景中，底层哈希不仅要作为普通签名算法中的安全原语，还需要被频繁写入有限域上的执行轨迹和代数约束。因此，哈希函数的“证明友好性”成为与传统软件安全性同等重要的设计指标。

从约束表达角度看，SHA-2、SHAKE 等传统哈希函数以位运算和字节置换为核心，在通用 CPU 上具有成熟实现，但在 STARK 或 SNARK 中通常需要将按位逻辑、移位和异或操作展开为有限域约束，导致证明成本较高。Poseidon 系列哈希则直接面向素域算术设计，原始 Poseidon 论文表明其可在零知识证明系统中以较少约束表示，并被用于 Merkle 树等证明密集型场景^[24]。Poseidon2 在此基础上进一步优化线性层，保留了

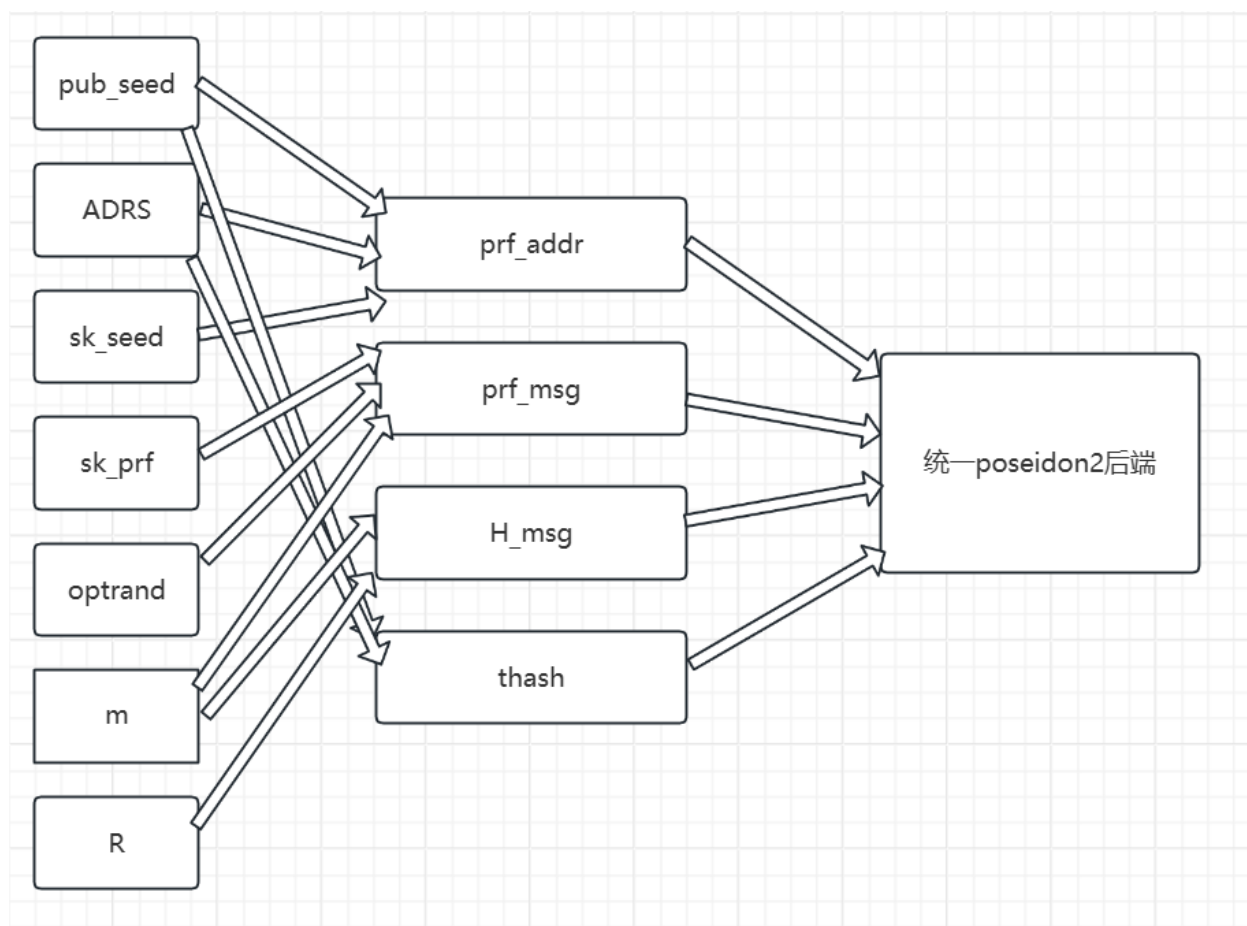


图 3.1 Poseidon2 替换 SPHINCS+底层哈希接口的总体结构

Poseidon 的基本设计路线，同时支持 sponge 和 compression 两种使用模式，适合将 SPHINCS+ 中大量树节点压缩与链式哈希调用统一到同一算术化后端^[25]。

选择 Poseidon2 而非仅采用 Poseidon 的原因在于，本文的瓶颈并不只来自哈希调用次数，也来自每次哈希在证明系统中的代数表示成本。Poseidon2 通过更高效的线性层减少乘法和线性组合开销，在不依赖 lookup argument 的前提下提升原生执行和电路表示效率^[25]。对于 SPHINCS+ 而言，WOTS+ 链、FORS 树、XMSS 子树以及 hypertree 认证路径都会引入大量哈希调用；若每个调用都能用更少的域运算和更稳定的固定参数配置表示，则后续 trace 统计、AIR 约束建模和 STARK 证明生成都会受益。

表 3.1 给出了本文在技术选型时关注的几类零知识友好哈希函数对比。需要说明的是，该表并不试图断言某一哈希在所有证明系统中全局最优，而是围绕本文的实现约束进行比较：底层证明系统采用 Winterfell/STARK 路线，当前实现不依赖 lookup argument；哈希后端需要服务 SPHINCS+ 的多类接口；工程实现需要与 C/Rust 中的有限域表示相衔接。

以太坊社区围绕 Poseidon 的讨论也从侧面说明了算术化友好哈希在 ZK 系统中的工程价值。EIP-5988 曾提出为 EVM 增加 Poseidon 预编译，理由包括其适合 ZK/validity rollup、能够服务 rollup 与 EVM 的互操作，并兼容 SNARK、STARK、Bulletproofs 等

表 3.1 零知识友好哈希函数的技术选型比较

哈希函数	主要特点	优势	局限	本文选型结论
Poseidon	面向素域算术的哈希函数	生态采用较广，已有较多分析和实现经验	原始线性层在部分场景中仍有优化空间 ^[25]	不作为当前主方案
Poseidon2	Poseidon 的优化版本，改进线性层并支持 compression 模式	无需 lookup 即具有较好的证明友好性和原生性能 ^[25]	相对 SHA-2、Keccak 等传统哈希仍较新，参数需固定说明	本文主方案
Rescue-Prime	采用正向与逆向幂映射的算术化友好设计	安全分析较系统，适合作保守对照 ^[23]	逆幂运算可能增加原生实现成本	不作为当前主方案
Griffin/Anemoi	新一代 AO/ZK 友好哈希	在部分 R1CS、AIR 或 Plonkish 场景中约束数量较低 ^[23]	工程采用和长期密码分析积累相对较少	不作为当前主方案
Monolith/Reinforced Concrete/Tip5	偏向 lookup 或特定证明系统优化	在支持 lookup 的系统中可能具有较高吞吐 ^[23]	与本文不依赖 lookup 的 Winterfell 路线不完全一致	不作为当前主方案
SHA-2/SHAKE/Keccak	标准化传统哈希	安全分析充分，标准化程度高	位运算结构不利于有限域约束表达	保留为标准安全基线

多类证明系统^[43]。该提案同时指出，Poseidon 相比 SHA-256、Keccak 等传统哈希仍较年轻，且不同系统存在参数多样性问题^[43]。因此，在面向 STARK 证明的 SPHINCS+ 实验实现中，以 Poseidon2 替换传统哈希后端以降低约束成本。

底层有限域选择 Goldilocks 域 $p = 2^{64} - 2^{32} + 1$ ，则主要出于 STARK 证明与系统实现的共同考虑。一方面，该素域可用 64 位整数自然承载，其特殊模数形式有利于快速约简，并适合 C 语言中以 `uint64_t` 表示域元素；另一方面， $p - 1$ 含有较大的 2^{32} 因子，能够提供丰富的二次幂单位根，从而适配 STARK/FRI 中常见的低度扩展、插值和 NTT/FFT 型多项式运算^[44]。本文第三章后续采用固定的 Poseidon2 参数配置，并以小端序将字节流映射到 Goldilocks lane，正是为了使普通签名后端、trace 记录和后续 AIR 约束共享一致的数据表示。

综上，Poseidon2 与 Goldilocks 的组合是服务于“签名结构保持 SPHINCS+ 语义、证明系统保持 STARK 友好表达”的整体目标。Poseidon2 降低哈希调用的算术化复杂度，Goldilocks 降低有限域运算和 FRI 多项式操作的工程成本；二者共同构成本文将 SPHINCS+ 引入零知识盲签名场景的底层技术基础。

3.1.2 替换后保留的 SPHINCS+语义

本文的 Poseidon2 替换不是对 SPHINCS+签名结构的重新设计，而是对底层哈希后端的实例化替换。具体保留关系如表 3.2 所示。ADRS 仍以 32 字节地址编码参与 prf_addr 和 thash；pub_seed 仍作为公开实例分离参数参与相关哈希；sk_seed 和 sk_prf 仍分别承担地址 PRF 与消息随机化 PRF 的密钥化来源；optrand 仍参与消息随机化；H_msg 仍按 R 、 PK 和消息 M 生成 FORS 摘要、子树索引与叶子索引。

表 3.2 Poseidon2 替换中保留的 SPHINCS+核心语义

保留对象	原 SPHINCS+作用	Poseidon2 替换后的处理方式
ADRS	区分层级、节点位置和调用类型	仍以 32 字节编码作为哈希输入
pub_seed	提供公开参数与实例分离	在 prf_addr 和 thash 输入前缀中保留
sk_seed	派生 WOTS+/FORS 秘密节点	作为 prf_addr 的私钥化输入保留
sk_prf	生成消息相关随机值 R	作为 PRF_msg 的密钥化输入保留
optrand	对冲签名随机化	与 sk_prf 和消息共同输入 Poseidon2
digest/tree/leaf_idx	选择 FORS 消息和超树位置	仍由 H_msg 输出流切分得到

3.2 Poseidon2 后端与域分离设计

本文实现中的 Poseidon2 后端采用 64 位 Goldilocks 域，状态宽度 $t = 12$ ，容量字数 capacity=6，速率字数 rate=6，全轮数 $R_F = 8$ ，部分轮数 $R_P = 22$ 。该配置不是针对某一个接口的单点最优，而是作为所有 SPHINCS+底层哈希接口共享的固定后端参数配置。统一固定参数配置有利于保持实现可复现、trace 统计口径一致，并减少为不同接口维护多套 Poseidon2 参数所带来的工程复杂度。

```

#define SPX_POSEIDON2_FIELD_BITS 64
#define SPX_POSEIDON2_T 12
#define SPX_POSEIDON2_CAPACITY_WORDS 6
#define SPX_POSEIDON2_RATE_WORDS (SPX_POSEIDON2_T - SPX_POSEIDON2_CAPACITY_WORDS)
#define SPX_POSEIDON2_RATE_BYTES (SPX_POSEIDON2_RATE_WORDS * sizeof(uint64_t))
#define SPX_POSEIDON2_RF 8
#define SPX_POSEIDON2_RP 22

```

代码 3.1 poseidon2 相关参数定义

从零知识证明角度看，Goldilocks 域与 uint64_t lane 表示天然匹配，使字节流到域元

素的映射较为直接，并避免在 C 端引入复杂的大整数运算。状态宽度 $t=12$ 与 $rate=6$ 、 $capacity=6$ 的组合则在吞吐量和证明成本之间取得折中：较大的 $rate$ 可以减少长输入吸收次数，而不过度增大状态宽度又可控制每轮 $trace$ 和约束规模。

3.2.1 增量海绵接口

Poseidon2 后端以增量海绵接口对外提供可变长度哈希能力。接口包括初始化、吸收、结束填充和挤出四个阶段。初始化阶段首先吸收 1 字节域标签；吸收阶段按字节流接收输入，再由底层以小端序映射为 64 位 lane；结束阶段采用 pad_{10*1} 填充；挤出阶段输出所需字节数。

```
void poseidon2_inc_init(spx_poseidon2_inc_ctx *ctx, spx_poseidon2_domain domain_tag)
{
    memset(ctx, 0, sizeof(*ctx));
    ctx->domain_tag = (uint8_t)domain_tag;
    poseidon2_inc_absorb(ctx, &ctx->domain_tag, 1);
}

void poseidon2_inc_finalize(spx_poseidon2_inc_ctx *ctx)
{
    ctx->absorb_buf[ctx->absorb_pos] ^= 0x01u;
    ctx->absorb_buf[SPX_POSEIDON2_RATE_BYTES - 1] ^= 0x80u;
    p2_absorb_block(ctx, ctx->absorb_buf);
}
```

代码 3.2 Poseidon2 增量接口的初始化与最终填充

上述设计的关键在于：域标签不是以人工字符串前缀的形式散落在各接口中，而是在海绵初始化时被明确注入状态。这样既减少了编码不一致风险，也便于后续 $trace$ 记录和约束系统根据域标签直接识别调用语义。

3.2.2 域分离标签

由于 SPHINCS+ 在不同位置复用哈希原语，如果不做域分离，不同语义的输入可能落入同一哈希命名空间，增加实现混淆与安全分析难度。本文实现为地址 PRF、消息随机化、消息哈希以及树哈希中的 F 、 H 、 T_l 分别设置独立域标签，如表 3.3 所示。

3.3 四类底层哈希接口的适配

本节从接口语义和输入编码两个层面说明四类底层哈希接口的 Poseidon2 适配。四类接口的共同目标是保持 SPHINCS+ 原有地址、种子和消息绑定语义不变，同时把底层哈希调用统一为可追踪、可算术化的 Poseidon2 sponge 调用。对应实现代码集中列于附

录 A.1。

表 3.3 Poseidon2 后端中使用的主要域分离标签

域标签	取值	对应接口语义
SPX_P2_DOMAIN_PRF_ADDR	0x01	地址相关 PRF，用于 WOTS+/FORS 秘密值派生
SPX_P2_DOMAIN_GEN_MESSAGE_RANDOM	0x02	消息随机化 PRF，生成 R
SPX_P2_DOMAIN_HASH_MESSAGE	0x03	消息摘要 H_msg，输出 digest/tree/leaf_idx
SPX_P2_DOMAIN_THASH_F	0x11	单输入块树哈希 F
SPX_P2_DOMAIN_THASH_H	0x12	双输入块树哈希 H
SPX_P2_DOMAIN_THASH_TL	0x13	多输入块树哈希 T ₁

3.3.1 prf_addr 接口

prf_addr 用于根据公开种子、地址和私钥种子生成地址相关的伪随机值，是 WOTS+ 和 FORS 秘密节点派生过程的重要组成部分。本文实现将输入编码为 $\text{pub_seed} \parallel \text{ADRS} \parallel \text{sk_seed}$ ，并在 Poseidon2 中使用 PRF_ADDR 域标签。其形式化定义可写为：

$$\text{prf_addr}_{p_2}(\text{pub_seed}, \text{ADRS}, \text{sk_seed}) = \text{Poseidon2}_{\text{domain}}(\text{PRF_ADDR}, \text{pub_seed} \parallel \text{ADRS} \parallel \text{sk_seed})$$

该接口完整保留了 SPHINCS+ 中“公开实例参数、结构地址、私有种子”的三元绑定关系。pub_seed 保证不同公钥实例下同一地址不会产生相同命名空间，ADRS 绑定节点位置和用途，sk_seed 则提供私钥化来源。

3.3.2 PRF_msg 接口

PRF_msg 用于生成签名随机值 R。本文实现中对应函数为 gen_message_random，其输入为 sk_prf、optrand 和消息 M，输出长度为 SPX_N 字节。其定义可表示为：

$$\text{PRF_msg}_{p_2}(\text{sk_prf}, \text{optrand}, M) = \text{Poseidon2}_{\text{domain}}(\text{GEN_MESSAGE_RANDOM}, \text{sk_prf} \parallel \text{optrand} \parallel M)$$

这里保留了 SPHINCS+ 原有的密钥化与随机化机制。sk_prf 使 R 依赖签名者私钥材料，optrand 提供对冲随机性，消息 M 直接参与输入，从而避免同一密钥下不同消息共享相同随机化值。该设计也与第二章所述 Pure 模式中的双遍处理问题相衔接：R 依赖完整消息，因此 H_msg 阶段必须在 R 生成后再次处理消息。

3.3.3 H_msg 接口

H_msg 负责将 R、公钥和消息映射为 FORS 摘要、超树子树索引和叶子索引。本文实现保持 SPHINCS+ 输出切分语义不变，只是将底层输出流改为 Poseidon2 海绵挤出结果。

其定义为：

$$H_{msg_{p2}}(R, PK, M) = \text{Poseidon2}_{domain}(\text{HASH_MESSAGE}, R \parallel PK \parallel M)$$

Poseidon2 输出的 SPX_DGST_BYTES 字节缓冲区随后被依次切分为三部分：前 SPX_FORSS_MSG_BYTES 字节作为 FORS 消息摘要；接下来的 SPX_TREE_BYTES 字节解释为 tree，并按 SPX_TREE_BITS 掩码；最后 SPX_LEAF_BYTES 字节解释为 leaf_idx，并按 SPX_LEAF_BITS 掩码。

3.3.4 thash 接口

thash 是 SPHINCS+ 中可调哈希抽象的核心接口，用于实现 WOTS+ 链、FORS 树和 XMSS 树中的压缩操作。本文实现将输入统一编码为 $\text{pub_seed} \parallel \text{ADRS} \parallel \text{in}$ ，并根据 inblocks 选择不同域标签：inblocks = 1 时对应 F，inblocks = 2 时对应 H，inblocks ≥ 3 时对应 T₁。

$$\begin{aligned} THASH_{p2}(\text{pub_seed}, \text{ADRS}, X_1, \dots, X_s) = \\ \text{Poseidon2}_{domain}(THASH_semantic(s), \text{pub_seed} \parallel \text{ADRS} \parallel X_1 \parallel \dots \parallel X_s) \end{aligned}$$

按 inblocks 进行语义路由的好处在于：一方面，F、H 和 T₁ 作为不同树哈希语义不被混入同一域；另一方面，输入编码逻辑保持统一，便于代码维护、trace 统计和后续约束系统复用。对于 STARK 证明而言，统一输入布局减少了解析分支，仅需依据 domain_tag 和 inblocks 区分调用语义，接口总结如表 3.4。

3.4 工程实现细节

除了接口层替换外，Poseidon2 后端还需要处理字节流与有限域元素之间的映射、轮函数实现、trace 记录与约束统计等工程问题。这些细节决定了替换方案是否能够被后续 STARK 证明系统稳定消费，而不只是完成普通签名和验证。

3.4.1 字节到 Goldilocks 域元素的映射

本文实现采用小端序将输入字节打包为 64 位 lane，再映射到 Goldilocks 域元素。该方式与 C 语言中的 uint64_t 表示直接对齐，实现简单且易于复现。在 trace 适配代码中，还提供了将字节编码为 lane 数组的辅助接口，用于记录每次 Poseidon2 调用的输入规模。

```
int spx_p2_encode_bytes_to_lanes(uint64_t *out_lanes, size_t *out_count,
                                const uint8_t *in_bytes, size_t in_len);
```

代码 3.3 字节编码到 lane 接口

这种设计把签名后端与证明后端统一到同一数据表示上：普通签名阶段处理的是字节数组，证明统计阶段则可以进一步解释为有限域 lane 序列，从而为后续 AIR 约束或 trace

表格生成提供直接入口。

表 3.4 四类 Poseidon2 适配接口的输入编码

接口	域标签	输入编码	输出用途
prf_addr	PRF_ADDR	pub_seed ADRS sk_seed	地址相关伪随机值
PRF_msg	GEN_MESSAGE_RANDOM	sk_prf optrand M	签名随机值 R
H_msg	HASH_MESSAGE	R PK M	digest、tree、leaf_idx
thash-F	THASH_F	pub_seed ADRS X	单块链式压缩
thash-H	THASH_H	pub_seed ADRS X ₁ X ₂	二叉树节点压缩
thash-T ₁	THASH_TL	pub_seed ADRS X ₁ ... X _s	多块 WOTS+公钥压缩

3.4.2 轮函数与固定参数配置

Poseidon2 置换由前半段全轮、中间部分轮和后半段全轮组成。实现中包含 Goldilocks 域加法、乘法、 x^7 S-box、外部混合层、内部混合层和完整置换函数。本文采用固定 $R_F = 8$ 、 $R_P = 22$ 的参数配置，主要原因是保持参数重搜实验的统一基线。若在重搜 SPHINCS+ 参数的同时改变 Poseidon2 轮数和状态结构，则实验结果将难以区分是由签名参数变化导致，还是由哈希后端参数变化导致。

因此，本章将该固定参数配置视为工程基线，而非脱离具体仓库上下文的全局最优断言。后续工作仍可在固定 SPHINCS+ 参数后进一步搜索 Poseidon2 内部参数配置，以寻找证明成本与安全裕度之间更优的组合。

3.4.3 trace 与约束统计适配

为了服务 STARK 证明场景，本文实现为每次 Poseidon2 调用记录 domain_tag、输入字节长度、输出字节长度、ADRS、输入 lane 数量和输出 lane 数量等信息。该记录机制使得签名流程中的哈希调用可以被统一统计，并进一步转换为 witness_rows、trace_calls 和证明时间等指标。

3.5 安全性讨论与表述边界

Poseidon2 替换后的安全性需要从两个层面理解。第一，在 SPHINCS+ 结构层面，本文尽量保留原方案中节点地址、public seed、密钥化输入、消息随机化和输出切分等关键语义，从而使替换后的接口仍对应原有安全分析中的功能角色。第二，在具体实例化层面，Poseidon2 作为具体哈希函数，其安全性依赖所选有限域、状态宽度、轮数、S-box 指数和轮常量等参数，需要结合已知代数攻击和统计攻击进行评估。

更严格的安全论证需要进一步形式化 Poseidon2 实例化后的可调哈希函数族，并分析

其在多目标、不同 tweak 和量子访问模型下是否满足 SPHINCS+安全归约所需的性质。本文当前工作重点是工程替换、参数重搜和证明友好性评估，因此将这一问题作为后续理论分析方向。

3.6 本章小结

本章给出了将 Poseidon2 替换为 SPHINCS+底层哈希后端的设计与实现方案。该方案不改变 SPHINCS+上层签名结构,而是通过统一的 Poseidon2 海绵接口和显式域分离标签,分别适配 prf_addr、PRF_msg、H_msg 和 thash 四类底层接口。在适配过程中,ADRS、pub_seed、sk_seed、sk_prf、optrand 以及 H_msg 输出切分语义均被保留,从而使替换后的实现保持 SPHINCS+原有的结构边界。

第四章 基于 Fischlin 框架的盲签名方案设计与实现

4.1 技术选型、设计目标与符号约定

4.1.1 选择 Fischlin 框架的动机

本章选择 Fischlin 框架作为盲签名协议的上层结构，根本原因在于 SPHINCS+ 缺少可直接用于盲化和去盲化的代数结构。经典盲签名方案往往依赖 RSA、离散对数或双线性群中的可随机化性质，用户可以在代数关系中隐藏待签消息，并在签名完成后通过相应变换得到有效签名。SPHINCS+ 则是无状态哈希基签名方案，其签名由 FORS、WOTS+、XMSS 和超树认证路径组成，核心操作是伪随机派生、链式哈希和树节点压缩^[8]。这种结构具有保守的后量子安全基础，但并不天然支持类似 RSA 盲签中的乘法盲化。

在这一约束下，直接修改 SPHINCS+ 内部签名流程以获得盲性并不是稳妥路径。若试图在 WOTS+ 链或 FORS 选择过程中引入盲化因子，不仅会破坏原有地址、随机化和树结构语义，还可能使安全分析脱离 SPHINCS+ 已有的哈希基框架。相比之下，Fischlin 盲签名框架采用“承诺-签名-证明”的通用结构：用户先对消息生成承诺，签发者只对承诺进行普通签名，最终由用户给出零知识证明，证明自己知道该承诺的打开以及签发者对该承诺的有效签名^[31]。该框架把底层签名算法作为黑盒调用，因此特别适合缺少可随机化代数结构的哈希基签名。

在本文方案中，Fischlin 框架的作用是将盲性与签名算法本身解耦。签发者看到的是承诺 $c = \text{Com}(m; r)$ ，而不是消息 m ；底层 SPHINCS+ 只需对 c 进行普通签名，不需要理解或支持盲化变换；最终展示对象的有效性由证明系统保证。由此，盲性主要来自承诺的隐藏性和证明的零知识性，不可伪造性则依赖底层签名对承诺的不可伪造性、承诺绑定性以及证明系统的可靠性。

4.1.2 选择 STARK 与 Winterfell 的动机

证明系统方面，本文选择 STARK 而非基于配对或椭圆曲线假设的传统 SNARK，主要是为了保持方案整体的后量子一致性。STARK 以透明设置、可扩展证明和基于哈希/编码理论的安全基础为主要特征，不需要可信设置，并且其安全假设通常不依赖会被 Shor 算法直接破坏的离散对数或整数分解问题^[41]。由于本文底层签名已经选择哈希基 SPHINCS+，若外层证明系统再引入非后量子的代数假设，则整体方案的后量子安全叙事会出现不一致。因此，STARK 更适合作为本文盲签名证明关系的后端。

从证明对象的形态看，Fischlin 关系也适合被表达为 STARK 中的确定性计算。验证者需要确信：承诺 c 与消息和随机数一致，签发者对 c 的签名有效，最终公开对象 c^* 与内部凭证一致，且展示语句中的公开上下文被正确绑定。这些条件均可转化为有限域上的状态转移、边界约束和一致性检查。STARK 中的 FRI 低度测试为此类大规模计算完整性证明提供了基础，其证明器与验证器复杂度适合处理哈希密集型计算^[42]。虽然 STARK 证

明体积通常大于部分 SNARK，但本文更关注后量子安全、透明设置和可工程复现性，因此接受这一性能权衡。

工程实现上，本文选择 Winterfell 作为 STARK 框架，是因为其提供了 Rust 实现的 prover/verifier、AIR 抽象、执行轨迹接口和证明参数配置，能够较好地承接本文 C 端签名实现与 Rust 端证明实现之间的分层需求。Winterfell 文档将证明流程概括为定义 AIR、生成执行轨迹、实现证明器并调用验证接口；其并发证明生成、证明大小和验证策略等工程接口也便于进行第五章中的实验统计。此外，Winterfell 的开源介绍强调其目标是降低 STARK 使用门槛，使开发者主要关注计算的代数中间表示，而不必从零实现完整的 FRI、Merkle 承诺和随机挑战流程^[45]。

因此，本章的技术路线可以概括为三个层次：Fischlin 框架解决 SPHINCS+ 不具备天然盲化代数结构的问题，STARK 保持证明系统的透明性和后量子安全取向，Winterfell 则提供可落地的工程框架。三者共同服务于本文的核心目标，即在尽量保留 SPHINCS+ 哈希基安全语义的前提下，构造一个能够被零知识证明系统表达和验证的盲签名实现。

4.1.3 设计目标与符号约定

本章的设计目标是在不依赖代数同态结构、且仅以哈希基后量子原语为底层的前提下，构造一个面向零知识证明友好场景的盲签名实例化方案。具体而言，本文同时要求：

协议语义严格对应 Fischlin 原始定义中的承诺、签发、证明生成与验证四步；

底层签名 $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ 由第三章构造的 Poseidon2 实例化 SPHINCS+ 方案承担；

承诺方案 Com 与最终公开对象 c^* 的构造统一基于 Poseidon2 海绵接口，以便与底层签名共享算术化基础；

非交互式证明系统 $\Pi = (P, V)$ 由 STARK 系统实例化，且对外稳定暴露公开输入与私有见证的数据布局。

为便于与第 2.7.2 节的形式化定义一一对照，本章统一使用 Fischlin 原始记号叙述协议，再补充其在仓库实现中的对应对象，对应关系如表 4.1 所示。

在本章余下部分，除非特别说明，否则均按照表 4.1 进行符号与对象间的等价替换。

4.2 协议描述

4.2.1 协议总体结构

按 Fischlin 原始定义，本文盲签名协议由用户、签发者与验证者三方在公共参考串 crs 下交互。协议核心可以概括为“先对承诺签名，再把‘知道承诺及其有效签名’这一事实封装为最终可公开验证的盲签对象”，具体步骤如下文。

表 4.1 Fischlin 原始记号与本文实现对象的对应关系

Fischlin 记号	实现对象	含义说明
m	m	待签名消息
r	r	承诺随机数
$c = \text{Com}(m; r)$	com	Poseidon2 承诺输出, 签发者可见
$\sigma \leftarrow \text{Sign}(sk, c)$	sigma_com	对承诺 com 的 SPHINCS+ 底层签名
c^*	sigma_C	最终公开展示对象 (ciphertext-like 载体)
π	pi_f	展示阶段的非交互式证明对象
x	$(pk_{\text{sig}}, pk_E, c, m_{\text{pub}}, \text{ctx}_{\text{pub}}, c^*)$	公开语句
w	(m, r, σ, ω_2)	私有见证
$\sigma_{\text{blind}} = (c^*, \pi)$	$(\text{sigma_C}, \text{pi_f})$	最终对外展示对象

4.2.2 承诺与签发阶段

承诺阶段, 用户对消息 m 选择承诺随机数 r , 并计算

$$c = \text{Com}(m; r)$$

本文以 Poseidon2 海绵在固定域分离标签下对 $m||r$ 的吸收-挤出结果作为 Com 的实例化输出。承诺值 c 而非明文消息 m 被发送至签发者, 这是盲签语义成立的前提, 也是本节方案与 Fischlin 原始描述保持一致的关键。

签发阶段, 签发者持有签名密钥 sk , 在接收到承诺 c 后调用第三章实例化的 SPHINCS+ 签名算法计算

$$\sigma \leftarrow \text{Sign}(sk, c)$$

需要强调, 本协议中的签发者始终对承诺对象 c 进行签名, 而不是绕过承诺层直接对消息 m 签名。这一约束在工程实现中通过将签发接口的输入显式限定为 com 承诺字节串得以强制执行。

4.2.3 证明生成阶段

用户在接收到 σ 后, 首先在本地完成底层验证

$$\text{Verify}(pk_{\text{sig}}, c, \sigma) \stackrel{?}{=} 1.$$

若底层签名验证失败, 协议立即中止; 若成功, 用户继续依据 (pk_E, c, σ) 与新引入的随机量 ω_2 构造最终公开对象

$$c^* = \text{Bind}(pk_E, c, \sigma; \omega_2),$$

并以公开语句 x 与私有见证 w 调用证明系统生成

$$\pi \leftarrow P(\text{crs}, x, w).$$

最终用户对外公开的盲签对象写作

$$\sigma_{\text{blind}} = (c^*, \pi).$$

4.2.4 验证阶段

验证者并不接触底层签名 σ ，仅在公开语句 x 上检查

$$V(\text{crs}, x, \pi) \stackrel{?}{=} 1.$$

由此可见，验证者所验证的是“是否存在某个见证 w 使得公开语句 x 上的关系 R 成立”，而不是“是否看到了某个显式公开的底层签名”。这一性质既保证了底层签名 σ 的隐藏，也使得展示验证层与 Fischlin 原始定义在语义上同构。

4.3 公开语句、私有见证与证明关系

为了使后续证明关系的写法严格对应原始定义，本节明确公开语句 x 、私有见证 w 与证明关系的内容。

4.3.1 公开语句

Fischlin 原始描述中，公开语句至少包含消息、签名公钥与最终公开对象。本文方案在此基础上将所有需要被验证者读到、且会在 **Bind** 与严格证明语句绑定关系中被使用的公开量统一纳入 x ，最终写作

$$x = (pk_{\text{sig}}, pk_E, c, m_{\text{pub}}, ctx_{\text{pub}}, c^*),$$

其中各分量含义如下：

- pk_{sig} 是底层 SPHINCS+ 签名公钥；
- pk_E 是构造 c^* 时引入的额外公开参数；
- c 是中间承诺，等同于实现中的 com ；
- m_{pub} 是展示时允许公开的消息部分；
- ctx_{pub} 是展示验证上下文（包括协议版本、域分离标签等）；
- c^* 是最终公开对象。

补入 pk_E 、 c 与 ctx_{pub} 是本文实例化的工程需要，而非对原始定义的偏离：在不存在此类公开参数的特例下，公开语句即退化为 Fischlin 原始的 (m, pk, c^*) 形式。

4.3.2 私有见证

按 Fischlin 原始描述，见证常写作 (c, r, σ) 。在本文实例化中，承诺 c 由 (m, r) 唯一可重建，而最终公开对象 c^* 的构造还引入了额外随机量 ω_2 ，因此本文将见证记为

$$w = (m, r, \sigma, \omega_2)$$

这一写法与原始定义在“知晓承诺打开与有效底层签名”这一核心要求上完全等价，仅是借助关系 $c = Com(m; r)$ 在证明内部恢复 c ，并显式纳入 ω_2 以匹配本文 *Bind* 函数的输入空间。

4.3.3 证明关系的形式化写法

将上述协议组合起来，证明系统所需证明的关系可以首先按 Fischlin 原始风格写为

$$R(x, w) = 1 \Leftrightarrow c = Com(m; r) \wedge Verify(pk_{sig}, c, \sigma) = 1 \wedge$$

c^* 与 (m, c, σ) 满足公开关联关系。

为给出具体可实现的语言定义，本文进一步将“公开关联关系”展开为最终公开对象的构造一致性与展示语句绑定，于是协议对应的语言可写作

$$\begin{aligned} L = \{ & (pk_{sig}, pk_E, c, m_{pub}, ctx_{pub}, c^*) : \exists (m, r, \sigma, \omega_2) \\ & s.t. c = Com(m; r) \wedge Verify(pk_{sig}, c, \sigma) = 1 \\ & \wedge c^* = Bind(pk_E, c, \sigma; \omega_2) \wedge StmtBind(m_{pub}, ctx_{pub}, c, c^*) = 1 \}. \end{aligned}$$

四项关系在原始定义视角下的角色分别是：

1. $c = Com(m; r)$ 对应承诺的正确打开；
2. $Verify(pk_{sig}, c, \sigma) = 1$ 对应底层签名的有效性；
3. $c^* = Bind(pk_E, c, \sigma; \omega_2)$ 对应最终公开对象与内部对象之间的构造一致性；
4. $StmtBind(m_{pub}, ctx_{pub}, c, c^*) = 1$ 是本文针对展示阶段引入的严格证明语句绑定约束，使展示语句中的公开消息与上下文真正进入证明语义。

4.4 工程对象与接口实现

4.4.1 协议主链的接口分层

仓库中的实现按照 Fischlin 流程的语义边界拆分为承诺请求、签发响应、凭证收束、展示证明生成与展示验证等接口。请求阶段形成承诺，签发阶段只对承诺对象签名，凭证阶段收束消息、打开随机数、签名与二次随机量，展示阶段再把公开语句与私有见证送入证明系统。完整 C 接口清单见附录 A.2。

由 Fischlin 视角看，上述四类接口正好对应协议的四个阶段： $c = Com(m; r)$ ， $\sigma \leftarrow Sign(sk, c)$ ，将 (m, r, σ, ω_2) 收束为后续展示所需的内部凭证， $P(crs, x, w)$ 与 $V(crs, x, \pi)$ 。上述接口保证了签发者始终只接触承诺值而非明文消息，验证者最终接收的是公开展示对象与证明对象。工程实现中还提供包装接口以便测试主链闭环，但该包装不改变协议边界，只是把承诺、签发与凭证收束组合为一次实验调用。

4.4.2 展示对象与内部凭证的结构体编码

最终公开对象 $\sigma_{\text{blind}} = (c^*, \pi)$ 在实现中的主要字段包括用于携带最终公开绑定结果的 `sigma_C`、用于保存 STARK 展示证明的 `pi_f`、以及需要在验证侧复读和绑定的公开消息与公开上下文。结构体定义见附录 A.2。

其中 `sigma_C` 与 `pi_f` 分别对应 Fischlin 记号中的 c^* 与 π ; `m_pub` 与 `public_ctx` 是展示阶段公开语句 x 中需要被验证者复读与绑定的部分。该结构体在协议外部仅用作验证侧可见对象容器，本身不包含任何私有见证。

与之配套的内部凭证显式保留了消息与承诺打开 (m, r) 、中间承诺 `com`、对承诺的底层签名 `sigma_com`、构造 c^* 所需的额外随机量 `omega2`，以及可选的哈希调用轨迹 `trace`。这一设计使得论文中“私有见证”的说法在工程上落实为分字段结构体，并为后续证明输入打包提供基础。

4.4.3 最终公开对象的构造

$c^* = \text{Bind}(pk_E, c, \sigma; \omega_2)$ 最终公开对象的构造采用“承诺前缀 + Poseidon2 绑定后缀”的形式：前缀直接携带中间承诺，后缀由公开加密键、承诺、底层签名与二次随机量按固定顺序吸收到独立域分离标签后挤出得到。核心实现片段见附录 A.2。

这一设计带来两点重要后果。其一，`sigma_C` 的前缀直接携带 `com`，使验证侧可以在不接触底层签名的前提下从 c^* 中恢复 Fischlin 关系所需的中间承诺 c ；其二，`sigma_C` 后缀并非任意附加字段，而是对 $(pk_E, c, \sigma, \omega_2)$ 的 Poseidon2 吸收-挤出结果，其顺序与编码均为 AIR 约束所固定。因此 $c^* = \text{Bind}(pk_E, c, \sigma; \omega_2)$ 不再只是抽象数学关系，而是具有可复现序列化语义的工程对象。

4.4.4 公开输入与私有见证的打包

展示证明生成的实现先检查内部凭证形状，再构造最终公开对象 c^* ，最后把公开语句与私有见证分别送入 FFI 输入结构体。字段打包片段见附录 A.2。

该映射与论文中的公开语句和私有见证定义一一对应：

$$x = (pk_{\text{sig}}, pk_E, c, m_{\text{pub}}, ctx_{\text{pub}}, c^*), \quad w = (m, r, \sigma, \omega_2)$$

每个数学符号都对应一个 FFI 字段，因此协议的对象边界在跨语言交互处不会被丢失或重命名。

4.5 STARK 证明系统的接口与 AIR 落地

4.5.1 跨语言 FFI 边界

证明系统 $\Pi = (P, V)$ 在工程上被切分为两层：C 侧负责协议组织、对象编码与入口检查，Rust 侧负责 STARK trace 构造、AIR 约束求值与 `proof` 序列化。FFI 接口暴露稳定 ABI，使展示层只依赖公开输入、私有见证和证明数据块三类边界对象；具体结构体与函

数声明见附录 A.2。

该层接口的工程意义在于：上层协议已经把数学对象 x, w, π 固定为稳定数据布局，因此后续即便 AIR 列结构演化，展示层也无需修改。

4.5.2 严格输入与见证关系预检

Rust 侧在真正构造 trace 之前执行严格预检，分别检查公开输入字段、公开对象形状以及 witness 与承诺之间的基本一致性。预检代码见附录 A.2。

这种预检并非替代 AIR 约束，而是证明入口的语义守卫：格式错误、公开对象与承诺不匹配、或 witness 不能打开承诺时，系统在生成 trace 前即拒绝。验证侧也会先核对 proof 头部元数据，再进入 Winterfell 验证，从而避免无关证明在格式合法时被错误接收。。

4.5.3 Trace 与公开输入的列布局

当前严格证明使用固定长度 trace，并为承诺字块、最终对象后缀、公开上下文和语句绑定关系预留列。基础常量与边界断言片段见附录 A.2。

该布局的核心思想是把公开语句中的承诺 c^* 、最终公开对象 x 、公开上下文和公开加密键切分为 Goldilocks 域上的若干字块，并通过边界断言固定到 trace 起止位置。这样，公开语句不只是验证函数的外部参数，而成为 AIR 可检查的状态边界。

因此，语句绑定不再只是应用层约定。任何对公开消息、公开上下文、最终对象或承诺的篡改，都会改变边界字块或派生关系，从而导致验证拒绝。

4.5.4 承诺与最终对象的证明化路径

进入 trace 构造前，Rust 后端会从 witness 重新计算 com，并依据 $(pk_E, c, \sigma, \omega_2)$ 推导 sigma_C 后缀所需的吸收块与终态，关键状态推导片段见附录 A.2。

由此，关系 $c = \text{Com}(m; r)$ 与 $c^* = \text{Bind}(pk_E, c, \sigma; \omega_2)$ 在 AIR 中获得了具体承载：witness 打开的 (m, r) 必须能恢复出公开承诺 com，sigma_C 后缀块排布必须与吸收顺序一致，且 sigma_C 的若干字块必须与公开语句中的 c^* 保持相等。

4.5.5 证明参数配置与验证策略

主证明采用统一的 Winterfell 参数配置，具体证明选项设置见附录 A.2。

验证侧采用 `AcceptableOptions::MinConjecturedSecurity(63)`。这一组合反映了两点工程现实：第一，主证明统一在 Goldilocks 域上运行，便于与 Poseidon2 模块共享字段语义；第二，验证策略与现有证明参数配置的实际可达安全估计相匹配，而非在抽象层笼统声明“后量子安全”。

4.5.6 完成边界与混合证明模型

依据上文给出的接口与 AIR 设计，本文当前实现可以概括为以下完成项与边界：

已经完成的内容包括：协议展示主链接口闭环； c^* 的可复现构造与公开承诺前缀绑定； x, w 到 FFI 公私输入的稳定映射；com、sigma_C 后缀、public_ctx 等关系的 AIR 边界断

言与状态链约束；以及 pi_f 的格式封装、头部语义标签和 Rust 证明/验证主路径。

尚未完全统一进入 AIR 的关键关系是

$$Verify(pk_{sig}, c, \sigma) = 1.$$

在关系迁移中，这一关系仍由本地预检守护：

```
if (spx_p2_verify_com(pub->pk, pub->com, wit->sigma_com) != 0) {
    return SPX_P2_FFI_STATUS_ERR_PROVE;
}
```

代码 4.1 进入 STARK 主链前的本地预检

因此，本文已经把 Fischlin 盲签名的对象边界、展示接口和部分关键绑定关系落实到了可运行的 STARK 实现中，但底层签名验证关系仍采用本地检查与 AIR 约束增强相结合的混合证明模型，没有将全部关系内生到统一 AIR 中。

4.6 本章小结

本章在第三章给出的 Poseidon2 实例化 SPHINCS+ 签名原语之上，按照 Fischlin 通用盲签名框架，完成了协议描述、证明关系形式化与工程实现三层设计。在协议层面，本章严格沿用 Fischlin 原始符号 $c, \sigma, c^*, \pi, x, w$ 给出承诺、签发、证明生成与验证四个阶段，并在不偏离原始定义的前提下，将 pk_E 、 ctx_{pub} 与 ω_2 自然地纳入公开语句与私有见证。在证明关系层面，本章将原始“知晓承诺打开与有效底层签名”的存在性关系展开为承诺正确性、底层签名有效性、最终对象构造一致性与展示语句绑定四项条件。在工程层面，本章说明了协议主链如何被拆分为 issue/finalize/show/verify 四类接口、最终公开对象如何在 Poseidon2 海绵下被可复现地构造、公开输入与私有见证如何被组织为跨语言稳定 ABI，以及 STARK 后端如何通过固定 trace、字块边界断言与严格预检对协议关系给出部分 AIR 承载。最后，本章对混合证明模型给出了事实表述：协议语义、对象边界、展示接口与部分关键绑定关系已经纳入 STARK 承载层；底层签名验证关系仍由本地保护检查与 AIR 约束增强联合保证，并作为后续工作的主要扩展方向。

第五章 实验设计与结果分析

第三章和第四章分别完成了 Poseidon2 对 SPHINCS+ 底层哈希接口的替换设计，以及基于 Fischlin 框架的盲签名方案实现。本章设计系统实验对前述设计进行验证，分别回答正确性、传统软件性能、协议主链可运行性、参数选择方法与零知识证明友好性五方面问题，并给出对应的实验结论。

5.1 实验目标与实验环境

5.1.1 实验目标

本章实验在以下五个层次上对全文设计进行验证。

1. Poseidon2 替换是否保持了 SPHINCS+ 上层结构语义并通过基础正确性测试。
2. 在普通 CPU 环境下，Poseidon2 替换前后的签名与验签性能差异。
3. 第四章所述 Fischlin 主链是否已工程闭环并具备对关键篡改的拒绝能力。
4. 面向 Poseidon2 与 STARK 的参数搜索流程能否给出合理候选。
5. Poseidon2 与 SHA2 在 STARK 电路视角下的真实约束规模与证明性能差距。

5.1.2 实验环境

为保证实验结果可复现，本章统一采用如下环境：实验平台为一台 64 位 x86 计算机，处理器为 AMD Ryzen 7 4800U with Radeon Graphics，主频 1.80 GHz，内存 16 GB；操作系统环境为 WSL/Linux；C 端使用 gcc 编译；Rust 端通过 release 模式构建 STARK 后端；普通签名/验签性能测量使用 sphincs-poseidon2-192s（初期设置为与 sphincs-sha2-192s 相同）与 sphincs-sha2-192s 两组参数集；Fischlin 主链与 STARK 证明实验默认基于 sphincs-poseidon2-192s 参数集运行。

实验任务可分为三类。

第一类是前置功能回归测试，用于确认密钥生成、签名、验签、FORS 子模块与 Poseidon2 适配层在实验前均能稳定运行；该类结果不作为本章主要性能结论。

第二类是协议主链与证明对象测试，重点记录展示证明对象大小、证明生成时间、验证时间以及公开语句绑定失败时的拒绝行为。

第三类是性能与证明统计测试，主要度量普通签名与验签耗时、展示证明阶段开销、STARK 证明大小、trace 与 witness 规模，以及 SHA2 与 Poseidon2 在 THASH 场景下的证明成本差异。

5.2 性能评价指标与实验口径

5.2.1 评价指标

本章实验以功能正确性测试视为方案实现成立的前置条件，在此基础上重点考察方案的可量化性能表现、证明开销以及与相关工作的对比情况。结合本文方案的设计目标，后续实验统一围绕以下几类指标展开：签名大小、普通 CPU 环境下的签名时间与验签时间、STARK 执行轨迹规模、证明大小、证明生成时间以及证明验证时间。其中，签名大小用于反映不同参数集下方案在存储与传输方面的开销；普通 CPU 签名时间与验签时间用于衡量方案在传统软件环境中的实际运行成本。STARK 执行轨迹规模、证明大小、证明生成时间和证明验证时间，则用于刻画方案在零知识证明环境中的算术化复杂度与证明系统代价，从而展示本文方案在“传统计算效率”与“证明友好性”之间的权衡关系。

5.2.2 数据口径

为保证不同实验之间具有可比性，本文对各项数据的统计口径作如下统一规定。

对于普通签名与验签实验，均采用多次重复运行后的中位数作为最终结果，以降低偶然性系统波动对测量值的影响。

对于 STARK 相关实验，统一记录轨迹宽度（`trace_width`）、轨迹长度（`trace_length`）、迁移约束数（`transition_constraints`）、边界断言数（`boundary_assertions`）、证明字节数（`proof_bytes`）、证明生成时间（`prove_ms`）与证明验证时间（`verify_ms`）。其中，轨迹宽度与轨迹长度用于描述执行轨迹的基本规模；迁移约束数与边界断言数用于反映 AIR 约束系统的复杂程度；证明字节数、证明生成时间和证明验证时间则分别对应证明对象的体积、生成开销与验证开销。

在参数搜索实验中，除上述指标外，还进一步记录签名字节数（`sig_bytes`）、见证行数（`witness_rows`）与端到端证明时间（`prove_e2e_ms`），以便从签名体积、见证规模和整体证明成本三个方面评估不同参数组合的综合表现。

上述数据口径能够较为直接地反映本文方案在传统软件执行环境与证明系统执行环境中的性能差异，并为后续与相关工作的比较提供统一依据

5.3 普通 CPU 环境下的传统签名与验签 benchmark

5.3.1 实验目的与方法

本节比较 `sphincs-sha2-192s` 与 `sphincs-poseidon2-192s` 在普通 CPU 环境下的签名与验签开销。实验通过统一的签名与验签性能测量流程分别记录两种后端的运行时间，并采用中位数（`median`）作为统计指标，以减小单次测量噪声影响。

5.3.2 实验结果

普通 CPU 下两种后端的中位时间统计如表 5.1 所示。

表 5.1 SHA2 与 Poseidon2 后端的普通签名/验签 benchmark (median)

方案	参数集	sign_us_median	verify_us_median
SHA2 基线	sphincs-sha2-192s	1 089 731.240	943.160
Poseidon2 替换	sphincs-poseidon2-192s	35 402 570.440	27 016.300

将上述微秒量换算为更直观的毫秒量级，则 SHA2 基线签名约为 1 089.731 ms、验签约 0.943 ms；Poseidon2 替换版签名约为 35 402.570 ms、验签约 27.016 ms。

5.4 Fischlin 主链的正确性与性能测试

5.4.1 前置正确性回归

第四章已经描述了完整的 Fischlin 主链 Commit -> Issue -> FinalizeCredential -> Show -> Verify。本节通过实验说明该主链不仅在接口上闭合，而且在端到端运行与篡改样本拒绝两个层面上都满足设计预期。

围绕第四章实现的 Fischlin 主链，本节将实验验证组织为四个层次。第一层是协议闭环测试，检查 Commit -> Issue -> FinalizeCredential -> Show -> Verify 是否能够在当前证明语句绑定路径下完整执行。第二层是角色与端到端测试，检查签发者、持证者与验证者三方交互是否闭合，以及盲签对象能否完成从承诺生成到展示验证的端到端收束。第三层是绑定与一致性测试，检查公开消息与上下文、执行轨迹重放与跨后端接受/拒绝结论是否保持一致。第四层是 STARK 统计测试，用于提取展示证明阶段的规模与时间指标。

按上述组织，本节实验的测试项、验证内容与记录口径如表 5.2 所示。

表 5.2 Fischlin 主链实验测试组织与记录口径

实验类别	工程入口	主要验证内容	记录口径
协议主链闭环	poseidon2_protocol_flow	最终证明语句绑定路径下 Commit -> Issue -> FinalizeCredential -> Show -> Verify 是否可运行	ACCEPT/REJECT 与主链日志
协议主链闭环	poseidon2_protocol_flow_statement_bound	证明语句绑定同源主链目标是否可稳定运行	ACCEPT/REJECT 与主链日志

规格一致性	poseidon2_fischlin_statement_spec	协议实现与当前规格口径是否一致	通过/失败日志
规格一致性	poseidon2_fischlin_statement_bound_spec	证明语句绑定规格路径与实现是否一致	通过/失败日志
角色交互	poseidon2_roles_interaction	签发者、持证者、验证者三方交互是否闭合	通过/失败日志与负例结果
端到端盲签闭环	poseidon2_fischlin_blind_e2e	两个上下文下的 show 对象是否均可正确生成	通过/失败日志与证明长度
公开语句绑定	poseidon2_statement_binding	m_pub、com 与上下文是否被一致绑定	通过/失败日志
trace 重放绑定	poseidon2_trace_replay_binding	不同 trace 或上下文下是否会发生错误接受	通过/失败日志
跨后端一致性	poseidon2_cross_backend_consistency	C 端与 Rust strict 后端是否给出一致结论	共同样本集接受/拒绝一致性
STARK 统计	poseidon2_stark_stats	提取 strict STARK 展示证明链的规模与时间指标	数值统计

上述测试的实际运行结果均通过，前置正确性回归完成。

5.4.2 主链与 STARK 后端的性能数据

在协议主链正确性验证之外，本节还使用 STARK 证明性能测试对当前 strict STARK 展示证明链进行了隔离统计，结果如表 5.3 所示。

表 5.3 的数值说明，当前 strict STARK 展示证明链已经不仅能够稳定输出证明对象，而且其规模和时间开销可以被稳定记录。尤其是证明大小、见证行数与端到端证明时间三项指标，为 5.5 节参数搜索中的多目标比较提供了可复现实验基础。

表 5.3 strict STARK 展示证明链统计结果

指标	数值
trace_calls	3011
trace_lanes	41144
witness_rows	44155
proof_bytes	73047
preprocess_ms	61.797
prove_core_ms	73.934
prove_e2e_ms	135.754
verify_ms	4.939
constraints	76
assertions	80
trace_width	52
trace_length	64

5.5 面向 Poseidon2 与 STARK 的参数搜索

5.5.1 不能直接沿用标准参数的原因

虽然本文保留 SPHINCS+ 的整体结构，但在底层哈希函数由 SHA2/SHAKE 类原语切换为 Poseidon2 后，参数选择的评价标准已经发生变化。标准 SPHINCS+ 参数主要面向传统软件环境下的签名尺寸、签名速度和验证速度，而本文还需要考虑哈希调用在 STARK 证明中的算术化成本，例如轨迹调用数、轨迹列数、见证行数和端到端证明时间等指标。

因此，原参数集在传统哈希后端下可能是合理折中，但在 Poseidon2 与 STARK 结合的场景下未必仍然是最优。特别是 FORS 参数 k 、 a ，超树高度 h ，层数 d 以及 Winternitz 参数 w 会共同影响签名大小、普通签名耗时和证明约束规模。本文采用重新枚举、静态过滤、安全代理评估、逐候选重编译 benchmark 以及 Pareto 分析的方式重新选择参数

5.5.2 搜索流程

仓库中的搜索流程在本文中统一记作 $M1 \rightarrow M2 \rightarrow M3 \rightarrow M4$ 四阶段：

第一阶段 $M1$ （结构枚举与静态过滤）：枚举参数轴 (n, h, d, k, a, w, q) ，并强制满足 $h \bmod d = 0$ 、 $tree_bits \leq 64$ 、 $leaf_bits \leq 32$ 等实现边界条件。

第二阶段 $M2$ （安全过滤）：采用代码仓库定义的工程化安全筛选口径，结合哈希位数、FORS 位数、安全预算位数与可宣称安全位数，剔除明显不满足目标安全位数的候选，具体而言，本文以 $\min(8n, ka)$ 作为组合层面的保守代理下界，并结合 Poseidon2 实例假

设下界与签名预算退化项，得到候选参数的声明安全位数。

第三阶段 M3（真实重编译与 benchmark）：对 M2 通过的候选逐一重编译并实测传统签名/验签，盲签名主链签名/验签时间，以及轨迹调用数、见证行数等指标。

第四阶段 M4（Pareto 前沿分析）：以签名大小、签名/验签时间与见证行数为四个最小化目标，计算 Pareto 前沿，并据此给出“最小签名”、“最快签名”、“最小约束”与“综合平衡”三类推荐参数。

5.5.3 实验结果

本轮 128 位安全级别参数搜索实验已经完成，结果汇总可统计得到：

表 5.4 当前 128 位档最终推荐参数

推荐口径	参数名称	参数	主要指标
最小时间	p2-128f	(n=16,h=66,d=22, k=22,a=6,w=16 ,q=65536)	sig_bytes=15856, sign_ms=1897.187, verify_ms=116.099, time_total_ms=2013.287, witness_rows=46472, prove_e2e_ms=63.200
最小约束	p2-128cs	(n=16,h=60,d=6, k=14,a=10,w=16 ,q=65536)	sig_bytes=6800, sign_ms=69905.622, verify_ms=32.852, time_total_ms=69938.474, witness_rows=19930, prove_e2e_ms=22.654
最小签名	p2-128s	(n=16,h=66,d=11, k=17,a=8,w=256 ,q=65536)	sig_bytes=6688, sign_ms=59696.380, verify_ms=495.159, time_total_ms=60191.539, witness_rows=45406, prove_e2e_ms=233.678
综合最优	p2-128	(n=16,h=64,d=8, k=22,a=6,w=16, q=65536)	sig_bytes=7984, sign_ms=21362.860, verify_ms=42.520, time_total_ms=21405.380, witness_rows=25585, prove_e2e_ms=25.882

综上，本文已经完成面向 Poseidon2 与 STARK 的 M1→M2→M3→M4 参数搜索闭环，并在 128 位安全级别上得到可复现的 Pareto 前沿与推荐参数；更高安全级别的参数组由于时间与批量测量成本限制，未纳入本轮实验数据收集范围，可在后续工作中继续扩展。

5.6 Poseidon2 与 SHA2 在 STARK 电路中的对比

5.6.1 实验目的与方法

本节是本文证明友好性论证的核心实验。本节比较 THASH 关系在真实 STARK 电路中的证明成本。实验的核心思想是：对 SHA2 与 Poseidon2 分别构造同口径的 thash 精确实现公开语句，再在各自的 STARK 电路中完成证明与验证，最终比较 trace 宽度、trace 长度、过渡约束数、边界断言数、证明大小以及证明/验证时间。

这里所谓“同口径语句”，是指两类后端都证明同一个函数关系：给定公钥种子、地址、输入块序列以及期望输出，证明某一确定的 thash 实例在指定输入块数下满足 $\text{expected_output} = \text{THASH}(\text{pub_seed}, \text{ADRS}, \text{input})$ 。

因此，本节实验是在同一语句形式下，将 SHA2 与 Poseidon2 分别写入各自的 THASH STARK 电路，并直接比较真实证明开销。考虑到 SPHINCS+ 中 $\text{thash}(\text{inblocks}=1)$ 与 $\text{thash}(\text{inblocks}=2)$ 分别对应最常见的 F 与 H 两类语义，本节重点报告这两个场景下的 STARK 对比结果。

5.6.2 THASH STARK 对比：F 场景

在 $\text{thash}(\text{inblocks}=1)$ 即 F 场景下，精确 STARK 度量结果如表 5.5 所示。

表 5.5 F 场景下 SHA2 与 Poseidon2 的真实 STARK 对比

指标	SHA2	Poseidon2
trace_width	65	15
trace_length	512	64
transition_constraints	112	17
boundary_assertions	3 998	21
proof_bytes	67 921	27 203
prove_ms	436.598	23.623
verify_ms	24.476	3.788
exact_round_rows	256	60

可以看到，在 F 场景下，Poseidon2 电路在列宽、长度、过渡约束数、边界断言数、证明大小及证明/验证时间六个维度上同时显著优于 SHA2。

5.6.3 THASH STARK 对比：H 场景

在 $\text{thash}(\text{inblocks}=2)$ 即 H 场景下，对比结果如表 5.6 所示。

表 5.6 H 场景下 SHA2 与 Poseidon2 的真实 STARK 对比

指标	SHA2	Poseidon2
trace_width	65	15
trace_length	1 024	128
transition_constraints	139	17
boundary_assertions	7 550	21
proof_bytes	72 119	30 949
prove_ms	1 599.820	45.407
verify_ms	69.147	4.753
exact_round_rows	640	90

H 场景下 Poseidon2 仍然全面优于 SHA2, 且在证明大小、证明时间与验证时间三项关键指标上的差距已超过一个数量级。

5.6.4 F/H/T₁ 调用占比

为说明 F/H 优势对整体证明系统的影响, 本文进一步统计了 SPHINCS+ 中不同 thash 调用类型的占比。verify 路径实测得到 thash_hist=1:2807, 2:301, 17:1, 51:7, 换算后 F 占 90.08%、H 占 9.66%、T₁ 合计占 0.26%。在签名路径上 F/H/T₁ 占比约为 91.36% /8.53% /0.11%; 密钥生成路径上则为 99.74% /0.13% /0.13%。F 与 H 两类调用主导了 SPHINCS+ 的树哈希热点, 且 F 占绝对多数。因此 Poseidon2 在 F/H 上的优势对整体证明系统并非边缘改进, 而具有决定性影响, 如图 5.1 所示。

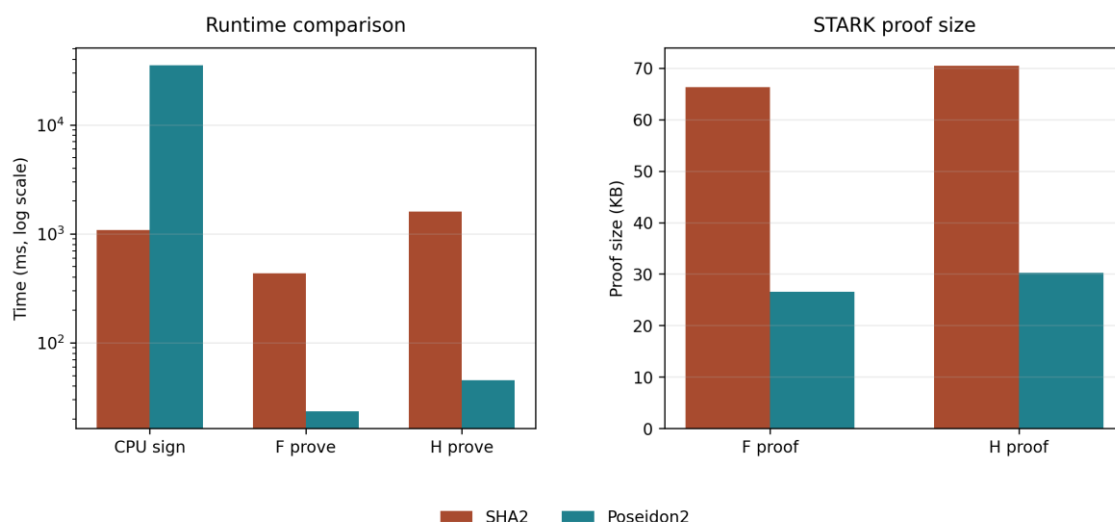


图 5.1 SHA2 与 Poseidon2 在 CPU 与 STARK 证明场景中的性能对比

5.7 相关工作性能对比

为使横向比较更清晰，本文将相关工作分为三类：第一类是 SPHINCS+ 官方参数，用于给出哈希基签名的签名大小与参数权衡基线^[46]；第二类是将 Bitcoin 优化版 SPHINCS+ 验签写入 Cairo/STARK 的工作，用于比较 Cairo steps、STARK 证明大小、证明时间和验证时间^[47]；第三类是后量子盲签名工作，用于比较盲签语义、交互轮次、签名或 transcript 大小与应用目标^{[48][49][50]}。由于这些工作分别面向标准签名、链上验签证明和盲签协议，本文先在表 5.7 中比较“是否盲签、是否 SPHINCS+、是否 STARK/Cairo、主要适用场景”，再在表 5.8 中单独列出签名大小、证明/通信大小、证明/签发时间和验证时间。

表 5.7 相关工作属性与适用场景对比

工作	是否盲签	是否 SPHINCS+	是否 Cairo/STARK	主要适用场景
本文方案	是	是，Poseidon2-SPHINCS+	是，Winterfell/STARK	盲签展示证明、公开语句绑定、证明友好哈希
SPHINCS+ 官方参数 ^[46]	否	是	否	哈希基签名规模与参数权衡基线
Bitcoin SPHINCS+ Cairo/STARK ^[47]	否	是，Bitcoin 优化版	是，Cairo + Stwo	SPHINCS+ 验签进入 Cairo/STARK 的直接对照
CDBS / Dilithium 盲签 ^[48]	是	否，Dilithium/格基	否	后量子盲签协议效率对照
LFSBS 前向安全盲签 ^[49]	是	否，SIS/格基	否	前向安全与应用型盲签功能对照
短轮次格基盲签 ^[50]	是	否，Ring/Module-SIS/LWE/NTRU	否	轮次、签名大小和通信大小对照

由于当前实现尚未将 SPHINCS+ 的完整验证关系内生到证明电路中，本文完整方案意义下的证明大小、证明生成时间与证明验证时间会超过现有实验结果。现有 STARK 数据仅反映外接展示证明承载层或局部哈希关系证明的实验开销。

表 5.8 相关工作的分指标性能对比

工作	签名大小	证明/通信大小	证明/签发时间	验证时间
本文方案	p2-128: 7,984 B	> 22 KB	>25.822 ms	> 0.249 ms
SPHINCS+ 官方参数 ^[46]	128s: 7,856 B; 128f: 17,088 B	无证明对象	给出参数权衡, 不是证明系统实验	给出普通验签复杂度基线
Bitcoin SPHINCS+ Cairo/STARK ^[47]	约 3,440 B	单签名证明约 4.8 MB; 目标聚合后约 100 KB	约 7 s; 238,000 Cairo steps	<20 ms
CDBS / Dilithium 盲签 ^[48]	公开材料未给出同口径签名大小	公开材料未给出同口径证明对象	盲签与验证平均约 657.65 μ s	包含在平均时间中
LFSBS 前向安全盲签 ^[49]	公开页面未列出同口径签名大小	公开页面未列出通信开销	较既有方案计算开销降低 22%–73%	未给出同口径验证时间
短轮次格基盲签 ^[50]	约 22 KB	transcript 最低约 60 KB	未给出硬件同口径时间	未给出硬件同口径时间

5.8 本章小结

本章围绕“Poseidon2 实例化 SPHINCS+ 盲签名方案是否正确、是否可运行、是否在证明场景下具备实际优势”这一主线，完成了功能验证、主链测试、参数搜索和 STARK 对比实验，并得出了较为清晰的总体结论。

第一，Fischlin 主链实验表明，本文方案已经实现了承诺、签发、凭证收束、展示与验证的完整闭环，并且能够对可公开消息、公开上下文等关键公开对象的篡改给出正确拒绝。在 strict STARK 路径下，系统已经能够生成展示证明对象，并给出相应指标，该方案已经达到“可运行、可验证、可统计”的工程状态。

第二，传统签名验签实验与 STARK 对比实验共同揭示了本文方案的核心权衡关系：Poseidon2 在传统软件签名/验签场景下明显慢于 SHA2，但在 STARK 证明电路中却能显著降低证明大小、证明时间与验证时间。以 F 场景为例，Poseidon2 相比 SHA2 将证明大小从 67,921 字节降至 27,203 字节，将证明时间从 436.598 ms 降至 23.623 ms，将验证时间从 24.476 ms 降至 3.788 ms，这直接证明了 Poseidon2 的优势主要体现于零知识证明友好性，而非传统 SPHINCS+ 签名验签性能。

第三，参数搜索实验表明，面向 Poseidon2 与 STARK 的 SPHINCS+ 参数不能直接沿用标准软件场景参数，而需要以签名大小、传统运行时间、见证规模和证明时间为联合目标重新搜索。实验流程已经在 128 位安全级别上得到可复现结果，并给出了“最小时间”“最小约束”“最小签名”和“综合平衡”四类推荐参数，这说明本文方案不仅可以运行，而

且具有进一步工程优化的空间。

综合全章实验,可以得到关键实验结论: Poseidon2 替换虽然牺牲了传统 CPU 上的签名性能,但成功换来了 SPHINCS+ 在 STARK 证明场景中的显著结构优势;在此基础上,本文进一步实现了一个能够完成展示证明、支持公开语句绑定并具备端到端闭环的 Fischlin 风格后量子盲签名方案。因而,本文工作的主要价值不在于提出一个传统软件环境下更快的 SPHINCS+ 变体,而在于验证了一条“将保守型哈希基后量子签名迁移到可证明验证环境”的可行工程路径。

第六章 总结与展望

6.1 全文工作总结

本文围绕"基于 Poseidon2 的 SPHINCS+ 盲签名方案设计与实现"这一题目,沿"底层原语 → 协议框架 → 证明系统 → 实验验证"的研究路线,构建了一个可运行、可验证的后量子盲签名工程化系统。

在底层原语层,本文在保留 SPHINCS+ 完整地址体系与树哈希语义的前提下,将 `prf_addr`、`PRF_msg`、`H_msg` 与 `thash` 四类底层接口统一替换为基于 Goldilocks 域的 Poseidon2 海绵实现,并通过显式域分离标签区分不同调用语义,从而在不改变上层签名结构的情况下,获得了一种算术化友好的 SPHINCS+ 签名后端。在参数层面,本文设计了四阶段参数搜索流程,针对签名时间、签名字节数、见证行数与 STARK 证明开销建立多目标 Pareto 分析,形成适用于 Poseidon2 + STARK 场景的候选参数体系,摆脱了对原始 SPHINCS+ 参数的直接照搬。

在协议框架层,本文以 Fischlin 通用盲签名框架为骨架,以 Poseidon2 实例化承诺 Com,以 SPHINCS+ 方案承担底层签名 Σ ,以 Winterfell STARK 系统承担证明系统 Π ,最终形成 $\sigma_{\text{blind}} = (c^*, \pi)$ 的完整承诺—签发—展示—验证主链,实现了签发者仅对中间承诺签名、验证者仅在公开语句上验证证明的完整工程流程闭环。

在证明系统层,本文将协议主链的对象边界、公开语句与私有见证组织为稳定的 C/Rust FFI 数据布局,并以 Winterfell + Goldilocks 为后端,将承诺打开关系、公开签名关键构造关系与严格证明语句绑定关系接入 STARK 主链 AIR。实验结果表明,在统一开销模型下, Poseidon2 STARK 证明路径约束数相对 SHA2 下降约 95.74%;在 F 与 H 两类 THASH 真实 STARK 对比中, Poseidon2 在轨迹宽度、过渡约束数、边界断言数、证明大小、证明时间与验证时间六项核心指标上全面优于 SHA2。与其它盲签方案的对比,也说明本方案在特定场景下具有使用性。上述结论印证了本文的核心主张: Poseidon2 在普通 CPU 环境下的传统签名性能不占优势,但其在 STARK 电路中所展现的结构优势,使其成为面向零知识证明的 SPHINCS+ 底层哈希后端的合理选择,以 Poseidon2 替换 SPHINCS+ 的盲签名方案,可以成为格基签名在零知识场景下的有效应用。

6.2 研究展望

本文围绕基于 Poseidon2 实例化 SPHINCS+ 的 Fischlin 盲签名与零知识展示验证方案进行了设计与实现,完成了从承诺、签发到展示、验证的基本语义闭环,验证了该方案在工程层面的可行性。但与此同时,本文也揭示出当前方案在证明系统内生度、形式安全论证完备性以及功能扩展性等方面仍存在进一步完善的空间。未来研究可围绕以下几个方向展开深入探索。

首先,在证明系统与验证性能方面,当前方案仍采用混合证明模型,部分底层签名验

证关系尚依赖本地预检与 AIR 边界断言联合保证，还没有完全纳入统一 AIR 约束体系。后续可进一步推动证明关系的完整内生性，将全部 thash 关系整体接入主证明，逐步消除对本地保护检查的依赖。在此基础上，还可引入递归 STARK 技术，对多个展示证明进行递归压缩与聚合，从而在大规模场景下降低验证开销，提升系统的整体可扩展性，推动方案从工程原型变成更具部署价值的实际系统。

其次，在理论安全性方面，本文当前最核心的缺口在于 Poseidon2 替换 SPHINCS+ 原生哈希接口后的形式安全证明尚不完备。SPHINCS+ 的安全性依赖底层哈希接口满足 PRF、SM-TCR、SM-DSPR、ITSR 等多种可证明安全性质，而现有 Poseidon2 研究主要聚焦于置换层面的统计安全性与抗代数攻击分析，尚未在 SPHINCS+ 所需的具体语义下建立完整归约。未来应围绕 prf_addr、PRF_msg、thash 与 H_msg 等接口逐一开展形式化分析，验证其是否满足 SPHINCS+ 安全证明所要求的密码学性质。同时，本文基于 Fischlin 框架实现了盲签名主链，但其盲不可伪造性与盲性安全仍有待严格的形式化证明；特别是盲不可伪造性的归约最终仍依赖底层 SPHINCS+ 在 Poseidon2 替换后的 EUF-CMA 安全性成立。因此，完善 Poseidon2 的形式安全分析，并进一步补全相关的安全归约，将是本文从工程实现走向理论严格性的关键步骤。

最后，在功能扩展方面，本文所构建的“承诺—签发—展示—验证”闭环已经具备改进为匿名凭证系统的良好基础。当前验证机制本身强调仅对公开语句进行验证、而不泄露底层消息与签名内容，因此与选择性披露匿名凭证的设计目标具有天然一致性。未来可在现有方案上进一步引入谓词证明层等，使持证者能够在不公开完整属性集的前提下，证明其属性满足特定条件，如年龄超过某一阈值、属于某一组织或具备某类资格等。由此，本文方案有望从单一盲签名原语扩展为属性选择性披露的后量子匿名凭证体系，为隐私保护身份认证、可验证资质证明等应用场景提供更完整的密码学支撑。

参考文献

- [1] Shor, P. W. (1994). Algorithms for quantum computation: Discrete logarithms and factoring. Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS '94), 124 – 134. <https://doi.org/10.1109/SFCS.1994.365700>
- [2] Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC '96), 212 – 219. <https://doi.org/10.1145/237814.237866>
- [3] National Institute of Standards and Technology. (2024). Module-lattice-based key-encapsulation mechanism standard (FIPS Publication 203). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.203>
- [4] National Institute of Standards and Technology. (2024). Module-lattice-based digital signature standard (FIPS Publication 204). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.204>
- [5] National Institute of Standards and Technology. (2024). Stateless hash-based digital signature standard (FIPS Publication 205). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.205>
- [6] Bernstein, D. J., & Lange, T. (2017). Post-quantum cryptography. Nature, 549(7671), 188 – 194. <https://doi.org/10.1038/nature23461>
- [7] Huelsing, A., Butin, D., Gazdag, S., Rijneveld, J., & Mohaisen, A. (2018). XMSS: eXtended Merkle signature scheme (RFC 8391). Internet Engineering Task Force. <https://doi.org/10.17487/RFC8391>
- [8] Bernstein, D. J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., & Schwabe, P. (2019). The SPHINCS+ signature framework. In Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19) (pp. 2129 – 2146). ACM. <https://doi.org/10.1145/3319535.3363229>
- [9] Barbosa, M., Dupressoir, F., Hülsing, A., Meijers, M., & Strub, P.-Y. (2024). A tight security proof for SPHINCS+, formally verified. In K.-M. Chung & Y. Sasaki (Eds.), Advances in Cryptology – ASIACRYPT 2024 (Lecture Notes in Computer Science, Vol. 15487, pp. 35 – 67). Springer. https://doi.org/10.1007/978-981-96-0894-2_2
- [10] Hülsing, A., Kudinov, M., Ronen, E., & Yorgev, E. (2023). SPHINCS+C: Compressing SPHINCS+ with (almost) no cost. In 2023 IEEE Symposium on Security and Privacy (SP 2023) (pp. 1435 – 1453). IEEE. <https://doi.org/10.1109/SP46215.2023.10179381>
- [11] Zhang, K., Cui, H., & Yu, Y. (2023). SPHINCS- α : Specification document [Submission to NIST Post-Quantum Cryptography Additional Signatures Round 1]. The SPHINCS- α

- Team. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-1/spec-files/sphincs-alpha-spec-web.pdf>
- [12] Niederhagen, R., Roth, J., & Wälde, J. (2022). Streaming SPHINCS+ for embedded devices using the example of TPMs. In L. Batina & J. Daemen (Eds.), Progress in Cryptology – AFRICACRYPT 2022 (Lecture Notes in Computer Science, Vol. 13503). Springer. https://doi.org/10.1007/978-3-031-17433-9_12
- [13] Kölbl, S., & Philipoom, J. (2024, April 11). A note on SPHINCS+ parameter sets [Conference presentation]. 5th NIST PQC Standardization Conference, National Institute of Standards and Technology. <https://csrc.nist.gov/csrc/media/Events/2024/fifth-pqc-standardization-conference/documents/papers/a-note-on-sphincs-plus-parameter-sets-pqc2024.pdf>
- [14] Zhou, Y., & Wang, Q. (2024). Dynamic optimization of SPHINCS+ signature algorithm parameters. Journal of Electronic Science and Technology, 22(4), Article 100257. <https://doi.org/10.1016/j.jnlest.2024.100257>
- [15] Kiktenko, E. O. (2020). SPHINCS+ post-quantum digital signature scheme with Streebog hash function. In AIP Conference Proceedings (Vol. 2241, p. 020014). AIP Publishing. <https://doi.org/10.1063/5.0011441>
- [16] 孙思维, 刘廷玉, 关志, 贺一凡, 胡磊, 景蔚群, 张立廷, 严含露. SPHINCS⁺-SM3: 基于 SM3 的无状态数字签名方案[J]. 密码学报, 2023, 10(6): 1135 – 1155.
- [17] Cherkaoui, A., Heravi, F., Kahrobaei, D., & Shahandashti, S. F. (2026). Spinel: A post-quantum signature scheme based on $SL_n(\mathbb{F}_p)$ hashing. arXiv. <https://doi.org/10.48550/arXiv.2602.09882>
- [18] Amiet, D., Leuenberger, L., Curiger, A., & Zbinden, P. (2020). FPGA-based SPHINCS+ implementations: Mind the glitch. In Proceedings of the 23rd Euromicro Conference on Digital System Design (DSD 2020) (pp. 229 – 237). IEEE. <https://doi.org/10.1109/DSD51259.2020.00046>
- [19] López-Valdivieso, J., & Cumplido, R. (2024). Design and implementation of hardware-software architecture based on hashes for SPHINCS+. ACM Transactions on Design Automation of Electronic Systems, 29(4), Article 56. <https://doi.org/10.1145/3653459>
- [20] Deshpande, S., Lee, Y., Karakuzu, C., Szefer, J., & Paek, Y. (2025). SPHINCSLET: An area-efficient accelerator for the full SPHINCS+ digital signature algorithm. ACM Transactions on Embedded Computing Systems. <https://doi.org/10.1145/3728469>
- [21] Zhou, Y., & Wang, Q. (2025). HERO-Sign: Hierarchical tuning and efficient compiler-time GPU optimizations for SPHINCS+ signature generation [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2512.23969>
- [22] Abi Ghanem, M., Courrier, M., Colombier, B., & Bossuet, L. (2026). Implementing

- SPHINCS+ on embedded systems: Ensuring performance in resource-constrained environments. In Proceedings of the European Congress of Embedded Real Time Systems (ERTS 2026). <https://hal.science/hal-05499913>
- [23] Guo, H., Feng, Y., Wu, C., Li, Z., & Xu, J. (2024). Benchmarking ZK-friendly hash functions and SNARK proving systems for EVM-compatible blockchains [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2409.01976>
- [24] Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., & Schofnegger, M. (2021). Poseidon: A new hash function for zero-knowledge proof systems. In Proceedings of the 30th USENIX Security Symposium (USENIX Security '21) (pp. 519 – 535). USENIX Association. <https://www.usenix.org/conference/usenixsecurity21/presentation/grassi>
- [25] Grassi, L., Khovratovich, D., & Schofnegger, M. (2023). Poseidon2: A faster version of the Poseidon hash function. In R. El Mabet, N. De Cannière, & A. Tian (Eds.), Progress in Cryptology – AFRICACRYPT 2023 (Lecture Notes in Computer Science, Vol. 14064, pp. 177 – 203). Springer. https://doi.org/10.1007/978-3-031-37679-5_8
- [26] Zhang, X., Steinfeld, R., Esgin, M. F., Liu, J. K., Liu, D., & Ruj, S. (2024). Loquat: A SNARK-friendly post-quantum signature based on the Legendre PRF with applications in ring and aggregate signatures. In L. Reyzin & D. Stebila (Eds.), Advances in Cryptology – CRYPTO 2024 (Lecture Notes in Computer Science, Vol. 14920). Springer. https://doi.org/10.1007/978-3-031-68376-3_1
- [27] Chen, L., Dong, C., Newton, C. J. P., & Wang, Y. (2024). Sphinx-in-the-head: Group signatures from symmetric primitives. ACM Transactions on Privacy and Security, 27(1), Article 11. <https://doi.org/10.1145/3638763>
- [28] Chaum, D. (1983). Blind signatures for untraceable payments. In D. Chaum, R. L. Rivest, & A. T. Sherman (Eds.), Advances in Cryptology: Proceedings of Crypto 82 (pp. 199 – 203). Plenum. https://doi.org/10.1007/978-1-4757-0602-4_18
- [29] Hauck, E., Kiltz, E., Loss, J., & Nguyen, N. K. (2020). Lattice-based blind signatures, revisited. In D. Micciancio & T. Ristenpart (Eds.), Advances in Cryptology – CRYPTO 2020 (Lecture Notes in Computer Science, Vol. 12171, pp. 500 – 529). Springer. https://doi.org/10.1007/978-3-030-56880-1_18
- [30] Bhoumik, S., Mitra, S., Sharma, R. R., & Namdeo, K. (2025). CSI-IBBS: Identity-based blind signature using CSIDH [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2509.06127>
- [31] Fischlin, M. (2006). Round-optimal composable blind signatures in the common reference string model. In C. Dwork (Ed.), Advances in Cryptology – CRYPTO 2006 (Lecture Notes in Computer Science, Vol. 4117, pp. 60 – 77). Springer. https://doi.org/10.1007/11818175_4
- [32] Ghadafi, E. (2013). Formalizing group blind signatures and practical constructions without

- random oracles. In C. Boyd & L. Simpson (Eds.), Information Security and Privacy – ACISP 2013 (Lecture Notes in Computer Science, Vol. 7959, pp. 379 – 394). Springer. https://doi.org/10.1007/978-3-642-39059-3_26
- [33] Katz, J., & Lindell, Y. (2020). Introduction to modern cryptography (3rd ed.). Chapman and Hall/CRC. <https://doi.org/10.1201/9781351133036>
- [34] Hülsing, A., Rijneveld, J., & Song, F. (2016). Mitigating multi-target attacks in hash-based signatures. In C.-M. Cheng, K.-M. Chung, G. Persiano, & B.-Y. Yang (Eds.), Public-Key Cryptography – PKC 2016 (Lecture Notes in Computer Science, Vol. 9614, pp. 387 – 416). Springer. https://doi.org/10.1007/978-3-662-49384-7_15
- [35] Bellare, M., & Rogaway, P. (1993). Random oracles are practical: A paradigm for designing efficient protocols. In Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS '93) (pp. 62 – 73). ACM. <https://doi.org/10.1145/168588.168596>
- [36] Boneh, D., Dagdelen, Ö., Fischlin, M., Lehmann, A., Schaffner, C., & Zhandry, M. (2011). Random oracles in a quantum world. In D. H. Lee & X. Wang (Eds.), Advances in Cryptology – ASIACRYPT 2011 (Lecture Notes in Computer Science, Vol. 7073, pp. 41 – 69). Springer. https://doi.org/10.1007/978-3-642-25385-0_3
- [37] Hülsing, A. (2013). W-OTS+ – Shorter signatures for hash-based signature schemes. In A. Youssef, A. Nitaj, & A. E. Hassanien (Eds.), Progress in Cryptology – AFRICACRYPT 2013 (Lecture Notes in Computer Science, Vol. 7918, pp. 173 – 188). Springer. https://doi.org/10.1007/978-3-642-38553-7_10
- [38] Halevi, S., & Krawczyk, H. (2006). Strengthening digital signatures via randomized hashing. In C. Dwork (Ed.), Advances in Cryptology – CRYPTO 2006 (Lecture Notes in Computer Science, Vol. 4117, pp. 41 – 59). Springer. https://doi.org/10.1007/11818175_3
- [39] Goldwasser, S., Micali, S., & Rackoff, C. (1989). The knowledge complexity of interactive proof systems. SIAM Journal on Computing, 18(1), 186 – 208. <https://doi.org/10.1137/0218012>
- [40] 李威翰, 张宗洋, 周子博, 邓燚. 简洁非交互零知识证明综述[J]. 密码学报, 2022, 9(3): 379 – 447.
- [41] Ben-Sasson, E., Bentov, I., Horesh, Y., & Riabzev, M. (2019). Scalable zero knowledge with no trusted setup. In A. Boldyreva & D. Micciancio (Eds.), Advances in Cryptology – CRYPTO 2019 (Lecture Notes in Computer Science, Vol. 11694, pp. 701 – 732). Springer. https://doi.org/10.1007/978-3-030-26954-8_23
- [42] Ben-Sasson, E., Bentov, I., Horesh, Y., & Riabzev, M. (2018). Fast Reed-Solomon interactive oracle proofs of proximity. In 45th International Colloquium on Automata, Languages, and Programming (ICALP 2018) (Leibniz International Proceedings in

- Informatics, Vol. 107, pp. 14:1 – 14:17). Schloss Dagstuhl – Leibniz-Zentrum für Informatik. <https://doi.org/10.4230/LIPIcs.ICALP.2018.14>
- [43] Bakhta, A., Ben Sasson, E., Levy, A., & Levit Gurevich, D. (2022). EIP-5988: Add Poseidon hash function precompile [Ethereum Improvement Proposal, Draft]. Ethereum Foundation. <https://eips.ethereum.org/EIPS/eip-5988>
- [44] Polygon Zero Team. (2022). Plonky2: Fast recursive arguments with PLONK and FRI [Technical report, Draft]. Polygon. <https://github.com/0xPolygonZero/plonky2/blob/main/plonky2/plonky2.pdf>
- [45] Khaburzaniya, I., Chalkias, K., Lewi, K., & Malvai, H. (2021, August 4). Open sourcing Winterfell: A STARK prover and verifier [Blog post]. Meta Engineering. <https://engineering.fb.com/2021/08/04/open-source/winterfell/>
- [46] Aumasson, J.-P., Bernstein, D. J., Beullens, W., Dobraunig, C., Eichlseder, M., Fluhrer, S., Gazdag, S.-L., Hülsing, A., Kampanakis, P., Kölbl, S., Lange, T., Lauridsen, M. M., Mendel, F., Niederhagen, R., Rechberger, C., Rijneveld, J., Schwabe, P., & Westerbaan, B. (2022). SPHINCS+: Submission to the NIST post-quantum cryptography project (v3.1). <https://sphincs.org/data/sphincs+-r3.1-specification.pdf>
- [47] AbdelStark. (2025). Quantum-resistant signatures for Bitcoin: A STARK experiment. HackMD. <https://hackmd.io/@AbdelStark/pq-btc-sig-stark>
- [48] Seno, M. E., Ramesh, J. V. N., Quraishi, A., et al. (2025). Post quantum-resistant blind signature scheme for consumer Internet of Things security. IEEE Transactions on Consumer Electronics. Advance online publication. <https://doi.org/10.1109/TCE.2025.3570340>
- [49] Xu, G., Fan, X., Chen, X., Liu, X., Li, Z., Mao, Y., & Zhang, K. (2025). An efficient anti-quantum blind signature with forward security for blockchain-enabled Internet of Medical Things. Computers, Materials & Continua, 82(2), 2293 – 2309. <https://doi.org/10.32604/cmc.2024.057882>
- [50] Beullens, W., Lyubashevsky, V., Nguyen, N. K., & Seiler, G. (2023). Lattice-based blind signatures: Short, efficient, and round-optimal. In Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23) (pp. 16 – 29). ACM. <https://doi.org/10.1145/3576915.3616613>

附录 A 代码实现清单

为避免正文过度展开底层实现细节，本文将第三章 Poseidon2 哈希替换与第四章 Fischlin 展示证明主链中的关键代码集中列于本附录。正文仅保留接口语义、对象边界和证明关系说明。

附录 A.1：第三章 Poseidon2 哈希替换相关代码

代码 A.1 Poseidon2 相关参数定义

```
#define SPX_POSEIDON2_FIELD_BITS 64
#define SPX_POSEIDON2_T 12
#define SPX_POSEIDON2_CAPACITY_WORDS 6
#define SPX_POSEIDON2_RATE_WORDS (SPX_POSEIDON2_T - SPX_POSEIDON2_CAPACITY_WORDS)
#define SPX_POSEIDON2_RATE_BYTES (SPX_POSEIDON2_RATE_WORDS * sizeof(uint64_t))
#define SPX_POSEIDON2_RF 8
#define SPX_POSEIDON2_RP 22
```

代码 A.2 Poseidon2 增量接口的初始化与最终填充

```
void poseidon2_inc_init(spx_poseidon2_inc_ctx *ctx, spx_poseidon2_domain domain_tag)
{
    memset(ctx, 0, sizeof(*ctx));
    ctx->domain_tag = (uint8_t)domain_tag;
    poseidon2_inc_absorb(ctx, &ctx->domain_tag, 1);
}

void poseidon2_inc_finalize(spx_poseidon2_inc_ctx *ctx)
{
    ctx->absorb_buf[ctx->absorb_pos] ^= 0x01u;
    ctx->absorb_buf[SPX_POSEIDON2_RATE_BYTES - 1] ^= 0x80u;
    p2_absorb_block(ctx, ctx->absorb_buf);
}
```

代码 A.3 prf_addr 接口

```
void prf_addr(unsigned char *out, const spx_ctx *ctx, const uint32_t addr8)
{
    unsigned char buf[SPX_P2_ENCODED_PRF_ADDR_BYTES];

    memcpy(buf, ctx->pub_seed, SPX_N);
    memcpy(buf + SPX_N, addr, SPX_ADDR_BYTES);
    memcpy(buf + SPX_N + SPX_ADDR_BYTES, ctx->sk_seed, SPX_N);

    poseidon2_hash_bytes_domain(out, SPX_N, SPX_P2_DOMAIN_PRF_ADDR, buf, sizeof(buf));
}
```

```
}

```

代码 A.4 PRF_msg 接口

```
void gen_message_random(unsigned char *R, const unsigned char *sk_prf,
                        const unsigned char *optrand,
                        const unsigned char *m, unsigned long long mlen,
                        const spx_ctx *ctx)
{
    spx_poseidon2_inc_ctx p2ctx;
    (void)ctx;

    poseidon2_inc_init(&p2ctx, SPX_P2_DOMAIN_GEN_MESSAGE_RANDOM);
    poseidon2_inc_absorb(&p2ctx, sk_prf, SPX_N);
    poseidon2_inc_absorb(&p2ctx, optrand, SPX_N);
    poseidon2_inc_absorb(&p2ctx, m, (size_t)mlen);
    poseidon2_inc_finalize(&p2ctx);
    poseidon2_inc_squeeze(R, SPX_N, &p2ctx);
}
```

代码 A.5 H_msg 接口

```
void hash_message(unsigned char *digest, uint64_t *tree, uint32_t *leaf_idx,
                  const unsigned char *R, const unsigned char *pk,
                  const unsigned char *m, unsigned long long mlen,
                  const spx_ctx *ctx)
{
    (void)ctx;

#define SPX_TREE_BITS (SPX_TREE_HEIGHT * (SPX_D - 1))
#define SPX_TREE_BYTES ((SPX_TREE_BITS + 7) / 8)
#define SPX_LEAF_BITS SPX_TREE_HEIGHT
#define SPX_LEAF_BYTES ((SPX_LEAF_BITS + 7) / 8)
#define SPX_DGST_BYTES (SPX_FORS_MSG_BYTES + SPX_TREE_BYTES + SPX_LEAF_BYTES)

    unsigned char buf[SPX_DGST_BYTES];
    unsigned char *bufp = buf;
    spx_poseidon2_inc_ctx p2ctx;

    poseidon2_inc_init(&p2ctx, SPX_P2_DOMAIN_HASH_MESSAGE);
    poseidon2_inc_absorb(&p2ctx, R, SPX_N);
    poseidon2_inc_absorb(&p2ctx, pk, SPX_PK_BYTES);
    poseidon2_inc_absorb(&p2ctx, m, (size_t)mlen);
    poseidon2_inc_finalize(&p2ctx);
    poseidon2_inc_squeeze(buf, SPX_DGST_BYTES, &p2ctx);

    memcpy(digest, bufp, SPX_FORS_MSG_BYTES);
}
```



```

    bufp += SPX_FORS_MSG_BYTES;

#if SPX_TREE_BITS > 64
    #error For given height and depth, 64 bits cannot represent all subtrees
#endif

    if (SPX_D == 1) {
        *tree = 0;
    } else {
        *tree = bytes_to_u11(bufp, SPX_TREE_BYTES);
        *tree &= (~(uint64_t)0) >> (64 - SPX_TREE_BITS);
    }
    bufp += SPX_TREE_BYTES;

    *leaf_idx = (uint32_t)bytes_to_u11(bufp, SPX_LEAF_BYTES);
    *leaf_idx &= (~(uint32_t)0) >> (32 - SPX_LEAF_BITS);

    spx_hash_profile_note_hash_message((size_t)SPX_N + (size_t)SPX_PK_BYTES + (size_t)m1en,
                                       SPX_DGST_BYTES);
}

```

代码 A.6 thash 接口

```

void thash(unsigned char *out, const unsigned char *in, unsigned int inblocks,
           const spx_ctx *ctx, uint32_t addr8)
{
    SPX_VLA(uint8_t, buf, SPX_P2_ENCODED_THASH_BYTES(inblocks));

    memcpy(buf, ctx->pub_seed, SPX_N);
    memcpy(buf + SPX_N, addr, SPX_ADDR_BYTES);
    memcpy(buf + SPX_N + SPX_ADDR_BYTES, in, inblocks * SPX_N);

    poseidon2_hash_thash_by_inblocks(out, SPX_N,
                                     buf, SPX_P2_ENCODED_THASH_BYTES(inblocks),
                                     inblocks);
}

```

代码 A.7 字节编码到 lane 接口

```

int spx_p2_encode_bytes_to_lanes(uint64_t *out_lanes, size_t *out_count,
                                const uint8_t *in_bytes, size_t in_len);

```

附录 A.2 第四章 Fischlin 展示证明与 STARK 接口相关代码

代码 A.8 协议主链的 C 接口

```
int spx_p2_issue_request(uint8_t out_com[SPX_N],
                        const uint8_t *m, size_t mlen,
                        const uint8_t *r, size_t rlen);

int spx_p2_issue_respond(spx_p2_issue_response_obj *out_resp,
                        const uint8_t *issuer_sk,
                        const spx_p2_issue_request_obj *req);

int spx_p2_finalize_credential(spx_p2_cred_internal *out_cred,
                              const spx_p2_issue_request_obj *req,
                              const spx_p2_issue_response_obj *resp,
                              const uint8_t *m, size_t mlen,
                              const uint8_t *r, size_t rlen,
                              const uint8_t *omega2, size_t omega2_len);

int spx_p2_protocol_show(spx_p2_show *out_show,
                        const uint8_t *pk_sig,
                        const uint8_t *pk_e, size_t pk_e_len,
                        const spx_p2_cred_internal *cred,
                        const uint8_t *public_ctx, size_t public_ctx_len);

int spx_p2_protocol_verify(const spx_p2_show *show,
                        const uint8_t *pk_sig,
                        const uint8_t *pk_e, size_t pk_e_len,
                        const uint8_t *m_pub, size_t m_pub_len);
```

代码 A.9 最终展示对象的 C 结构体编码

```
typedef struct {
    uint8_t sigma_C[SPX_P2_SIGMA_C_MAX_BYTES];
    size_t sigma_C_len;
    uint8_t m_pub[SPX_BYTES];
    size_t m_pub_len;
    uint8_t pi_f[SPX_P2_SHOW_PI_F_MAX_BYTES];
    size_t pi_f_len;
    uint8_t public_ctx[SPX_P2_SHOW_PUBLIC_CTX_MAX];
    size_t public_ctx_len;
} spx_p2_show;
```

代码 A.10 最终公开对象 c* 的构造

```
poseidon2_inc_init(&ctx, SPX_P2_DOMAIN_CUSTOM);
```

```
poseidon2_inc_absorb(&ctx, lbl_pke_ct, sizeof(lbl_pke_ct));
poseidon2_inc_absorb(&ctx, pk_e,      SPX_N);
poseidon2_inc_absorb(&ctx, com,      SPX_N);
poseidon2_inc_absorb(&ctx, sigma_com, SPX_BYTES);
poseidon2_inc_absorb(&ctx, omega2,   SPX_N);
poseidon2_inc_finalize(&ctx);
poseidon2_inc_squeeze(out_sigma_c + SPX_N, SPX_N, &ctx);

memcpy(out_sigma_c, com, SPX_N);
*out_sigma_c_len = 2u * SPX_N;
```

代码 A.11 公开输入与私有见证的字段打包

```
pub.pk      = pk_sig;
pub.pk_e    = pk_e;
pub.com     = cred->com;
pub.m_pub   = cred->m;
pub.public_ctx = public_ctx;
pub.sigma_c = out->sigma_C;

wit.sigma_com = cred->sigma_com;
wit.m         = cred->m;
wit.r         = cred->r;
wit.omega2    = cred->omega2;
```

代码 A.12 STARK 证明系统的稳定 FFI 接口

```
typedef struct {
    const uint8_t *pk;
    const uint8_t *pk_e;      size_t pk_e_len;
    const uint8_t *com;
    const uint8_t *m_pub;     size_t m_pub_len;
    const uint8_t *public_ctx; size_t public_ctx_len;
    const uint8_t *sigma_c;   size_t sigma_c_len;
} spx_p2_ffl_public_inputs;

typedef struct {
    const uint8_t *sigma_com;
    const uint8_t *m;       size_t mlen;
    const uint8_t *r;       size_t rlen;
    const uint8_t *omega2;  size_t omega2_len;
} spx_p2_ffl_private_witness;

int spx_p2_ffl_generate_pi_f(spx_p2_ffl_blob *out_proof,
                             const spx_p2_ffl_public_inputs *pub,
                             const spx_p2_ffl_private_witness *wit);
```

```
int spx_p2_ffi_verify_pi_f(const spx_p2_ffi_blob *proof,
                          const spx_p2_ffi_public_inputs *pub);
```

代码 A.13 进入 trace 构造前的严格预检

```
let strict_input_ret =
    spx_p2_rust_validate_strict_relation_inputs_v1(pub_inputs, wit, 1);
if strict_input_ret != SPX_P2_RUST_OK { return strict_input_ret; }

let strict_witness_ret =
    spx_p2_rust_validate_strict_witness_relation_v1(pub_inputs, wit);
if strict_witness_ret != SPX_P2_RUST_OK { return strict_witness_ret; }
```

代码 A.14 严格证明的 trace 基础参数

```
const TRACE_LEN: usize = 64;
const COM_LEN:  usize = 24;
const SIGMA_C_LIMBS: usize = 6;
const CIPHERTEXT_SUFFIX_BLOCK_COUNT: usize = 4;
const TRACE_WIDTH: usize =
    CIPHERTEXT_SUFFIX_STATE_COL_START + CIPHERTEXT_SUFFIX_CHAIN_COLS;
```

代码 A.15 公开语句字块进入 AIR 边界断言

```
Assertion::single(18, 0,      self.com_public_l0),
Assertion::single(18, last_step, self.com_public_l0),
Assertion::single(21, 0,      self.sigma_c_public_l0),
Assertion::single(21, last_step, self.sigma_c_public_l0),
Assertion::single(28, 0,      self.public_ctx_l0),
Assertion::single(28, last_step, self.public_ctx_l0),
Assertion::single(31, 0,      self.sigma_ctx_rel_l0),
Assertion::single(31, last_step, self.sigma_ctx_rel_l0),
```

代码 A.16 由 witness 推导 trace 关键状态

```
let mut com_from_witness = [0u8; SPX_N];
rust_commit_domain(&mut com_from_witness, m_wit, r_wit);

let (ciphertext_prev_prev_block,
    ciphertext_prev_block,
    ciphertext_final_block) =
    derive_ciphertext_suffix_blocks(pk_e, com, sigma_com_wit, omega2_wit)
        .ok_or(SPX_P2_RUST_ERR_INPUT)?;

let ciphertext_blocks =
    build_ciphertext_absorb_blocks(pk_e, com, sigma_com_wit, omega2_wit)
        .ok_or(SPX_P2_RUST_ERR_INPUT)?;
```

代码 A.17 Winterfell ProofOptions 配置

```
fn options_96bits() -> ProofOptions {  
    ProofOptions::new(32, 8, 0, FieldExtension::None,  
        8, 31,  
        BatchingMethod::Linear,  
        BatchingMethod::Linear)  
}
```

致谢

转眼之间本科时光已经接近尾声，本论文的完成，不仅是我个人在本科学习阶段的一次总结，也离不开老师的指导，家人的支持与朋友的陪伴，在此，我谨向所有关心与帮助我的所有人表达衷心的感谢。

首先我要感谢我的指导老师关志老师，从论文选题、资料收集、研究思路的梳理，到论文内容的修改与完善，关老师始终给予我耐心细致的指导。在我选题出现方向偏差时，老师也给予我及时的纠正，帮助我不断理清思路、改进不足。关老师严谨认真的治学态度、踏实负责的工作作风以及对学生的关心与鼓励，都让我受益匪浅，也将会成为我今后学习和生活中不断前行的动力。

同时，我也要感谢大学期间所有任课老师。正是老师们的辛勤付出和悉心教导，使我逐步掌握了专业知识，拓宽了学术视野，让我在种种专业中找到了适合自己的方向，并具备了完成本论文所需的基本能力。课堂上的启发、课后的指导，都为我的专业技能成长奠定了坚实基础。

此外，我要感谢身边的同学和朋友。在论文写作和日常生活中，大家给予了我许多帮助与陪伴。学习上的讨论交流和生活中的关心与照顾，都让我的大学时光更加充实而温暖。

最后，我要感谢我的家人。感谢他们一直以来对我的理解、包容与支持。在我迷茫时给予鼓励，在我疲惫时给予关怀，是他们默默的陪伴和无私的爱，让我能够安心学习并顺利完成学业。

谨以此文，向所有帮助、支持和陪伴过我的人致以最真挚的感谢。

北京大学学位论文原创性声明和使用授权说明

原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品或成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律结果由本人承担。

论文作者签名：刘震清

日期：2026 年 5 月 14 日

学位论文使用授权说明

本人完全了解北京大学关于收集、保存、使用学位论文的规定，即：

- 按照学校要求提交学位论文的印刷本和电子版本；
- 学校有权保存学位论文的印刷本和电子版，并提供目录检索与阅览服务，在校园网上提供服务；
- 学校可以采用影印、缩印、数字化或其它复制手段保存论文；

论文作者签名：刘震清

导师签名：关志

日期：2026 年 5 月 14 日